

Emmanuelle SANCHEZ

Développement Web et Web Mobile

Mingle

Dossier Projet
Juillet 2024

→ 01	Introduction		p.03
	Compétence du référentiel couvertes	p.03	
	Présentation de l'équipe	p.03	
	Présentation du projet et cahier des charges	p.04	
→ 02	Pré Production		p.05
	Gestion du projet	p.05	
	Diagramme d'utilisation	p.07	
	Charte graphique, Wireframe et maquette	p.08	
→ 03	Base de données		p.11
	Modèle conceptuel de données	p.11	
	Modèle logique de données	p.12	
	Modèle physique de données	p.13	
	Langage de définition de données	p.14	
→ 04	Développement		p.17
	Structure du projet	p.17	
	Front-end	p.19	
	Back-end	p.41	
→ 05	Finalisation du projet		p.50
	Tests	p.50	
	Déploiement	p.51	
→ 06	Développement des compétences		p.53
→ 07	Conclusion et Annexes		p.54

01 – INTRODUCTION

Compétences du référentiel couvertes

Développer la partie front-end d'une application web ou web mobile sécurisée

- Installer et configurer son environnement de travail en fonction du projet web/web mobile
- Maquetter des interfaces utilisateur web ou web mobile
- Réaliser des interfaces utilisateur statiques web ou web mobile
- Développer la partie dynamique des interfaces utilisateur web ou web mobile

Développer la partie back-end d'une application web ou web mobile

- Mettre en place une base de données relationnelle
- Développer des composants d'accès aux données SQL et NoSQL
- Développer des composants métier coté serveur
- Documenter le déploiement d'une application dynamique web ou web mobile

Compétences transversales

- Apprendre en continu
- Mettre en œuvre une démarche de résolution de problème
- Communiquer en français et en anglais

Présentation de l'équipe

Le projet Mingle a été réalisé par une équipe de 5 membres :

- Renaud Mercier
- Manon Drozdowski
- Guillaume Martinez
- Mehdy Bouzid
- et moi même, Emmanuelle Sanchez

Nous nous sommes rencontrés dans le cadre de notre formation de Développeur Web et Web Mobile, et avons décidés de collaborer ensemble afin de créer le projet Mingle

01 – INTRODUCTION

Présentation du projet

Le projet

Notre projet Mingle consiste en la création d'une **plateforme collaborative**, visant à faciliter l'échange de services entre particuliers. De part cette initiative nous avons cherché à créer un réseau dynamique et solidaire pour répondre aux besoins diversifiés utilisateurs, dans un contexte économique difficile.

Cahier des charges

Les utilisateurs pourront publier et répondre à des demandes de services, créer des profils détaillés, et utiliser des filtres de recherche selon divers critères comme la localisation ou le domaine d'expertise.

Un système de messagerie intégré et un système commentaires des services seront également inclus pour garantir la qualité et la confiance au sein de la communauté. La vérification de l'identité des utilisateurs sera assurée par un scan de pièce d'identité lors de l'inscription.

Les principales fonctionnalités sont :

- *La création de Profils Utilisateur*, facilitant ainsi la recherche de partenaires pour des collaborations spécifiques.
- *La publication de Demandes/Offres de Services*
- *Le filtrage des Offres*, afin de faciliter la recherche et rendre notre application plus efficace.
- *Une messagerie intégrée*, pour permettre aux utilisateurs de communiquer facilement pour discuter des détails des services proposés ou demandés.
- *Un système de commentaires*, pour une application plus douce et inclusive, afin de garantir la qualité des échanges sur la plateforme et contribuer à renforcer la confiance au sein de la communauté.

02 – PRE PRODUCTION

Gestion du projet

Méthodes Agiles

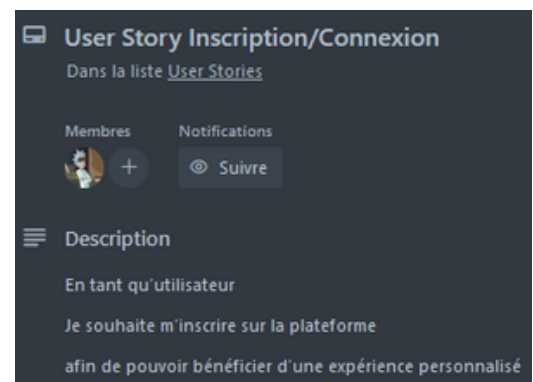
Afin de mener à bien le projet Mingle nous avons pris un temps de réflexion afin d'organiser notre méthode de travail. Celle-ci s'est inspirée des méthodes agiles :

- **Daily quotidiens** : Tous les matins nous avons pris 5/10min afin de faire le point sur ce qui avait été réalisé la veille, ce qui était prévu le jour même, ainsi que les difficultés rencontrées (voir annexes).
- **Sprint** : Nous avons organisé notre travail en itération de 1 à 2 semaines (par exemple 1 semaine pour réaliser les maquettes, un peu moins de 2 semaines pour la modélisation de la base de données ...)
- **Scrum Master** : Nous avons nommé un scrum master, à savoir moi même, afin d'organiser la communication entre nous.

Nous avons voulu, par cette approche flexible et itérative du développement, centrer notre projet sur la **collaboration** étroite entre les membres de l'équipe, la livraison rapide de valeur, et l'amélioration continue

Nous avons aussi créé des **User Story** afin d'aligner le travail de l'équipe sur les besoins des utilisateurs, et permettre une gestion flexible et efficace des priorités et des tâches dans cet environnement agile.

Exemple d'une User Story



Trello

Nous avons créé un espace de travail Trello (voir annexes), que nous avons alimenté quasi quotidiennement afin d'organiser et suivre l'avancement de notre travail.

Nous y avons recensé :

- Des liens utiles (un google drive, le Github du projet ...)
- Toutes les user stories
- La liste des fonctionnalités principales
- Les tâches "à faire", "en cours", "à tester" et "terminé"
- La liste des composants REACT à créer
- Les assets
- Et enfin la liste des features optionnelles

Cela nous a aidé à maintenir l'équipe synchronisée, à suivre le progrès du projet et à garantir que les objectifs sont atteints efficacement.

02 – PRE PRODUCTION

Gestion du projet

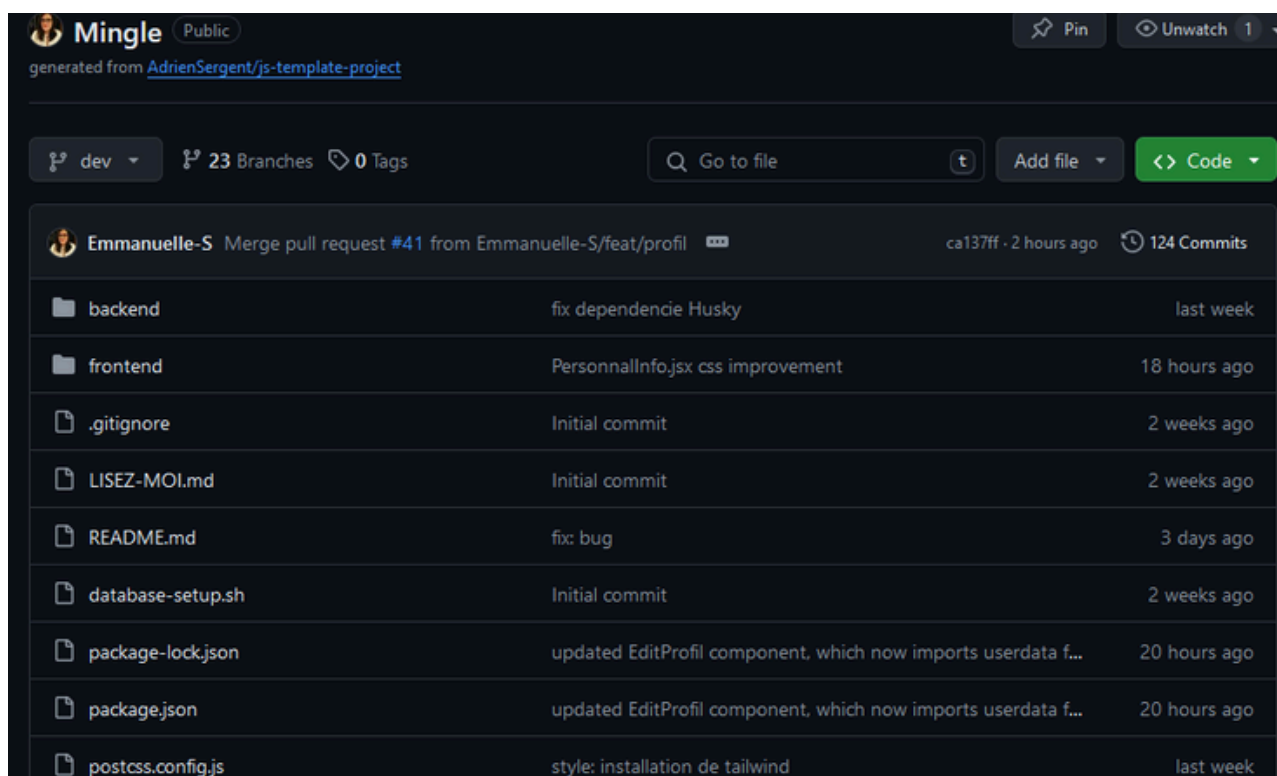
GitHub

Nous avons créé un espace commun sur GitHub, qui nous offre une plateforme robuste pour la gestion du code, la collaboration efficace, l'intégration continue et le suivi des tâches et des bugs. Cela nous a permis d'améliorer la qualité du code, de faciliter la communication et de garantir que le projet progresse de manière organisée et transparente.

Nous avons aussi beaucoup utilisé la pratique du versionning :

- Assurer la stabilité des déploiements
- Suivre les changements grâce à des commits détaillés
- Pouvoir revenir en arrière
- Travailler de façon simultanée sur différentes branches
- Réduire les conflits

Aperçu du versionning

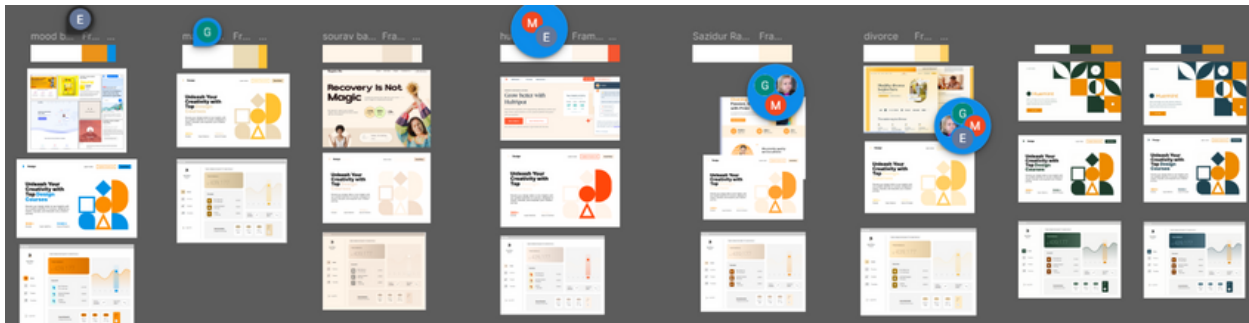


02 – PRE PRODUCTION

Charte graphique, Wireframe et maquette

Charte graphique

Afin de déterminer notre charte graphique nous avons créé un moodboard de plusieurs sites internet de services pour pouvoir affiner notre choix de couleurs. Nous les avons tous réunis sur l'outil Figma, et nous avons organisé un vote pour les couleurs qui nous plaisaient le plus.



Nous avons ensuite mélangé les nuanciers ayant remportés le plus de vote, et vérifiés que les couleurs correspondaient bien ensemble sur un spectre chromatique avant de nous décider sur la version finale

Choix final de la charte graphique



Wireframe

Les wireframes ont été créés sur l'outil Figma, sous forme d'**esquisses** en noir et blanc basiques. Ils nous ont servi à ébaucher une structure simple de l'application, à visualiser la disposition des différents éléments et à voir comment hiérarchiser l'information.

Cette étape incontournable nous a permis de vérifier que les éléments clés du cahier des charges seraient bien intégrés et facile d'accès pour l'utilisateur.

Nous avons fait une version desktop et une version mobile afin d'intégrer le responsive dès le début. Nous nous sommes réparti équitablement 4/5 wireframes par membre sur l'espace de travail commun Figma, ce qui nous a aussi permis de suivre l'avancement de chacun, et ainsi avoir une vue d'ensemble de l'application, et pas seulement de la partie qui nous était assignée.

J'ai pour ma part réalisé les wireframes de la page de connexion/inscription, ainsi que de toute la partie profil utilisateur (voir annexes).

02 – PRE PRODUCTION

Charte graphique, Wireframe et maquette

Maquette

Les maquettes ont été conçues dans le but de donner un **aperçu réaliste** de ce à quoi allait ressembler notre application finale.

Nous avons découpé notre site en plusieurs pages regroupant les différentes fonctionnalités, en essayant de nous rapprocher de ce qu'allaient être les futurs composants React (voir parti "Front-end" p.19), afin que chacun puisse développer la partie qu'il avait maqueté.

Nous y avons intégré la charte graphique commune, créé des composants communs (comme le header et le footer) pour unifier nos pages, et créé un prototypage afin de relier les pages entre elles (voir annexes).

Comme pour les wireframes, nous avons travaillé depuis un espace de travail commun sur Figma, et les maquettes ont été pensées en version desktop et mobile.

J'ai continué de gérer la partie connexion/inscription, ainsi que la partie profil utilisateur.

Liste des pages des maquettes

Communes :

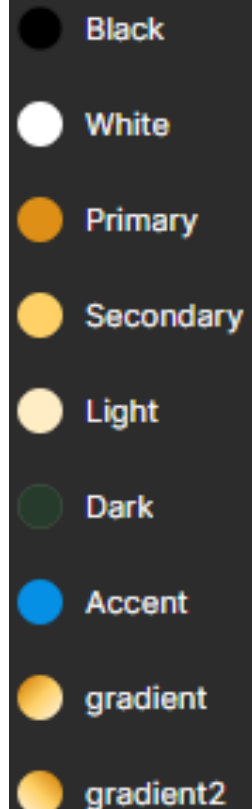
- Header
- Footer

Individuelles :

- Cookie
- Messagerie (ouverte et fermée)
- FAQ
- Détail service
- Connexion/Inscription
- Modifier profil
- Profil public
- Profil Privé
- Service
- Gestion des services
- A propos
- Liste de services
- Contact
- 404
- Accueil
- Publication de services
- Conditions d'utilisation
- Politique de confidentialité

Style commun

Color styles



02 – PRE PRODUCTION

Charte graphique, Wireframe et maquette

Maquette de la page de connexion/inscription version desktop

The desktop wireframe shows a page titled "Connexion/Inscription". The header includes a logo, navigation links (Accueil, Services, Publier, Messagerie), and a "Contactez Nous" button. The main content area features two side-by-side forms. The "Connexion" form on the left has fields for "Email" and "Mot de passe", a "Mot de passe oublié" link, and a "Se connecter" button. The "Inscription" form on the right has fields for "Nom d'utilisateur", "Email", "Mot de passe", and "Confirmation du mot de passe", a checkbox for "J'accepte les termes et conditions", and an "Inscription" button. The footer contains links for "FAQ / Aide", "Conditions d'utilisations", and "Politique de confidentialité".

Maquette de la page de connexion/inscription version mobile

The mobile wireframes are shown for "Android large" and consist of three panels. The first panel, titled "Inscription", contains fields for "Nom d'utilisateur", "Email", "Mot de passe", and "Confirmation du mot de passe", a checkbox for "J'accepte les termes et conditions", and an "Inscription" button. The second panel, titled "Connexion", contains fields for "Email" and "Mot de passe", a "Mot de passe oublié" link, and a "Se connecter" button. The third panel, titled "Mingle", features two large buttons: "Connexion" and "Inscription". All three panels share a common footer with links for "FAQ / Aide", "Conditions d'utilisations", and "Politique de confidentialité".

03 – BASE DE DONNEES

La base de données a été modélisée avec la **méthode MERISE**. C'est une méthode de conception, de développement et de réalisation de projets informatiques, très utilisée en France. Elle permet la séparation des données et les traitements à effectuer en plusieurs modèles conceptuels et physiques.

Nous avons d'abord travaillé sur papier pour poser nos idées, puis sur le logiciel Draw.io pour modéliser notre base de données.

Modèle conceptuel de données

Il permet d'établir une représentation claire des données et définit les dépendances fonctionnelles de ces données entre elles. Il nous a permis de commencer à visualiser la structure de notre base de données, et comment nous allons implémenter les données dans notre Base de Données.

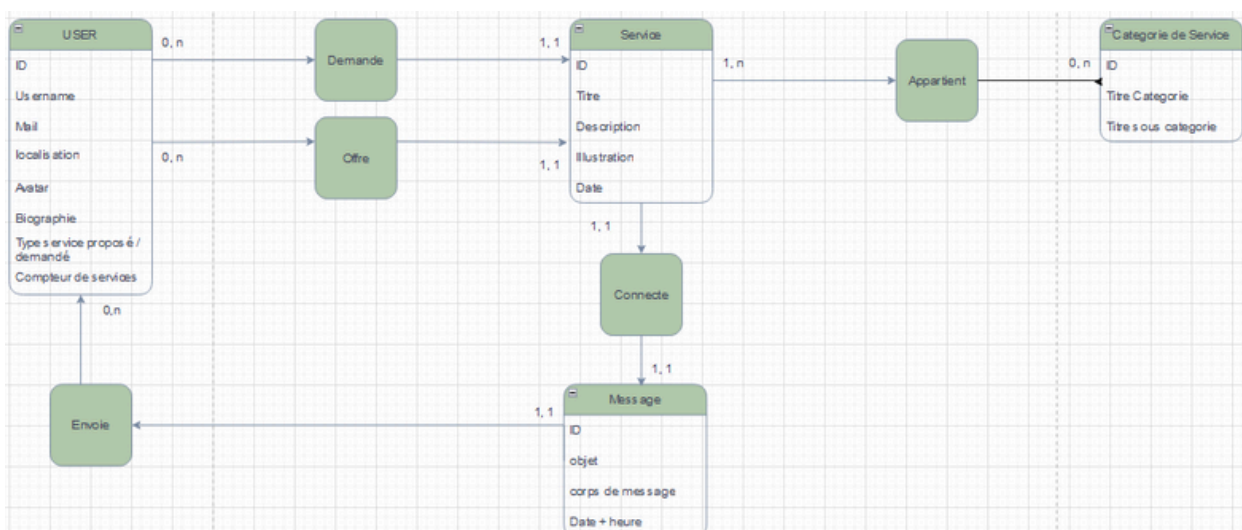
Étapes

1/ Nous avons déterminé quelles seraient les **entités** de notre base de données (c'est à dire quels éléments auraient les mêmes caractéristiques), quelles **propriétés** caractériseraient ces entités ainsi que leurs **identifiants**.

2/ Nous avons ensuite déterminé les **relations** entre les différentes entités. Nous les avons caractérisé par un verbe d'action, caractérisant la relation entre ces entités.

3/ Enfin, nous avons déterminé quelles seraient les **cardinalités**, c'est à dire les indications minimum et maximum des **liens** entre deux entités : 0, 1 ou n (plusieurs)

Schéma du modèle conceptuel de données



03 – BASE DE DONNEES

Modèle conceptuel de données

Voici comment nous pouvons lire ce graphique :

- Un Utilisateur(user) peut offrir ou demander un ou plusieurs Service.
- Un Utilisateur(user) peut envoyer un ou plusieurs Message
- Un Service appartient à une ou plusieurs Catégorie de Service
- Un Service est connecté à l'envoi /réception d'un Message

Modèle logique de données

Il est la traduction du modèle conceptuel de données en y ajoutant notamment les clés primaires et clés étrangères.

Dans ce modèle nous avons transformés les entités et leurs attributs, ainsi que leurs relations en Table, faisant un pas de plus vers un modèle physique de Base de Données.

Cette étape nous a permis de nous rendre compte de certaines redondances ou incohérence.

Étapes

1/ Nous n'avons gardé que les entités, en supprimant les relations entre elles.

2/ Nous avons gardé les plus grandes cardinalités des deux côtés pour déterminer le type de relation. Deux issues sont possibles :

→ One To One (1, 1), c'est à dire que chaque enregistrement dans la table A est associé à un seul enregistrement dans la table B, et chaque enregistrement dans la table B est associé à un seul enregistrement dans la table A,
ou Many To One (1, n), c'est à dire qu'un enregistrement dans la table A peut être lié à plusieurs enregistrements dans la table B, mais chaque enregistrement dans la table B est lié à un seul enregistrement dans la table A

→ Many To Many (n, m), c'est à dire qu'un enregistrement dans la table A peut être lié à plusieurs enregistrements dans la table B, et chaque enregistrement dans la table B peut également être lié à plusieurs enregistrements dans la table A

3/ Deux possibilités selon l'issue :

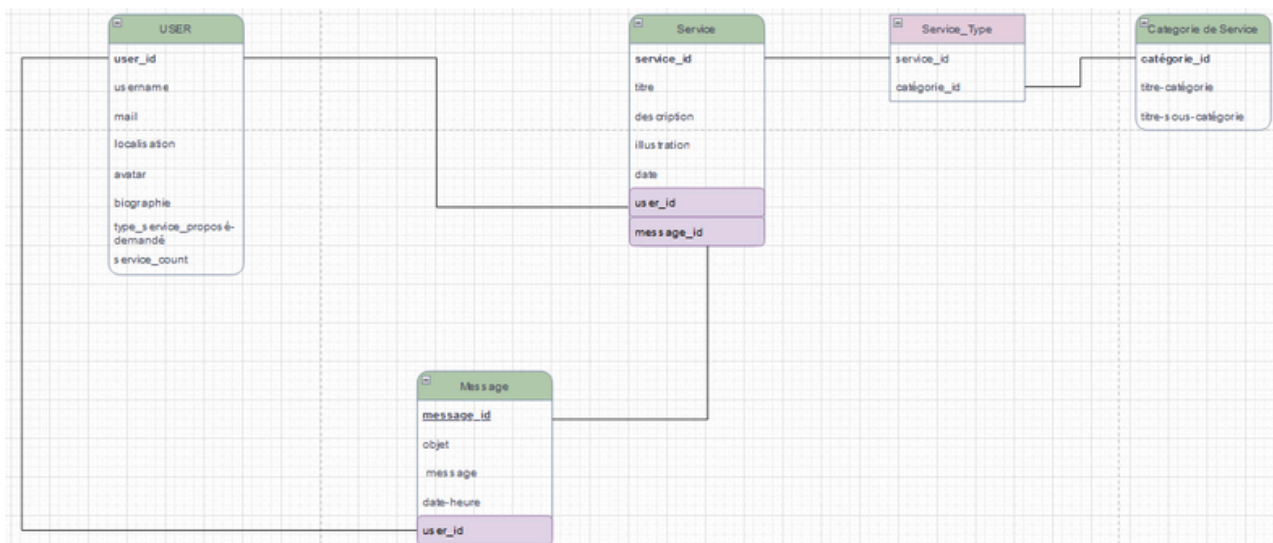
→ One To One (1, 1) ou Many To One (1, n) : Nous ajoutons un clé étrangère du côté du "1", qui prend généralement la valeur d'un champs unique de la table (ici l'ID)

→ Many To Many (n, m) : Donne lieu à la création d'une table de jointure contenant les deux clé étrangères (ce couple de clés peut suffire comme clé primaire de la table de jointure)

03 – BASE DE DONNEES

Modèle logique de données

Schéma du modèle logique de données



Voici comment nous pouvons lire ce graphique :

- Nous avons rajouté la clé étrangère **"user_id"** provenant de la table **User** dans la table **Message**
- Nous avons rajouté les clé étrangère **"user_id"** et **"message_id"** dans la table **Service**, provenant respectivement des table **User** et **Message**
- Nous avons crée une table de jointure **"Service_Type"**, provenant de la relation "many to many" entre les table **Service** et **Catégorie de Service**

Modèle physique de données

Il est la **représentation finale** d'une base de données, prenant en compte les spécificités du système de gestion de base de données utilisé, ici MySQL.

Ce **niveau physique** va tenir compte des particularités de MySQL, comme les type de données, les contraintes d'intégrités (qui sont des règles à suivre quand des enregistrements de tables reliées sont mis à jour ou supprimés)

Cette structure physique est optimisée pour le stockage, la récupération et la manipulation des données.

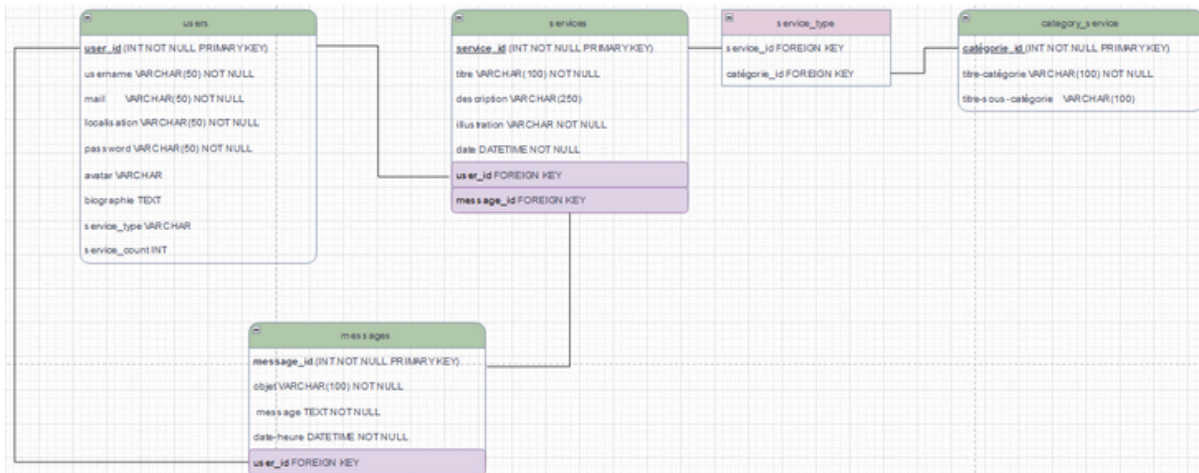
Cette étape nous a demandé une certaine concentration car une conception soignée de MPD assure des performances optimales et une gestion efficace des données.

Par exemple, nous avons oublié l'option `ON DELETE CASCADE` lors de la conception du MPD, ce qui nous a ralenti lors de nos tests un peu plus tard (nous l'avons rajouté directement sur MySQL)

03 – BASE DE DONNEES

Modèle physique de données

Schéma du modèle physique de données



Voici comment nous pouvons lire ce graphique, avec l'exemple de la table **User** :
Nous avons attribué des contraintes à chaque colonne (anciennement attribut) de la table :

- NOT NULL : la colonne ne peut pas contenir de valeur nulle
- INT : est utilisé pour stocker des nombres entiers dans une base de données
- PRIMARY KEY : identifie de manière unique chaque enregistrement dans une table
- VARCHAR : utilisé pour stocker des chaînes de caractères de longueur variable
- TEXT : utilisé pour stocker de grandes chaînes de caractères

Langage de définition de données

Nous avons décidé d'articuler notre base de données en utilisant MySQL, qui est le SGBDR fondamental, **phpMyAdmin** pour administrer et **Alwaysdata** pour héberger les bases de données MySQL.

Nous avons utilisés des **requêtes SQL** pour communiquer avec MySQL et manipuler les données.

Et enfin **Postman** pour tester notre API en envoyant des requêtes HTTP (GET, POST, PUT, DELETE, etc.) aux serveurs web et en inspectant les réponses (voir annexes p.50)

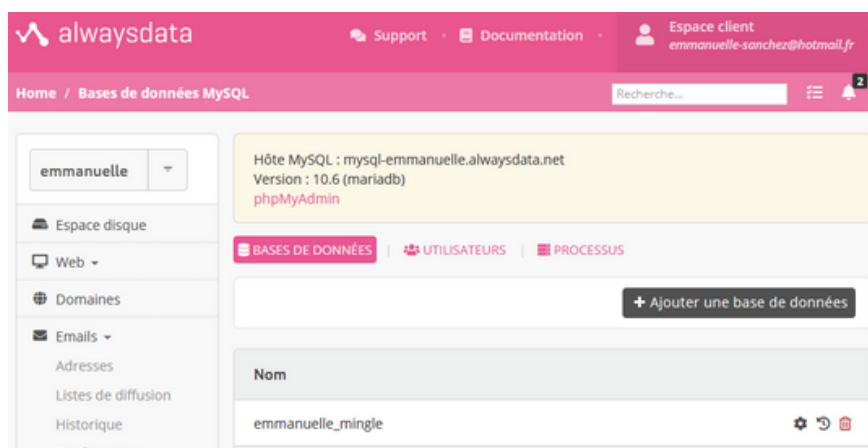
03 – BASE DE DONNEES

Langage de définition de données

AlwaysData

L'espace de base de donnée a été créé (et hébergé) via AlwaysData. Ce service d'hébergement web et de bases de données nous a permis de déployer et de gérer la base de données grâce à son interface complète. Une fois créée, nous avons pu assigner à tous les membres de l'équipe un compte afin que chacun puisse y avoir accès.

Dashboard AlwaysData



Liste des membres

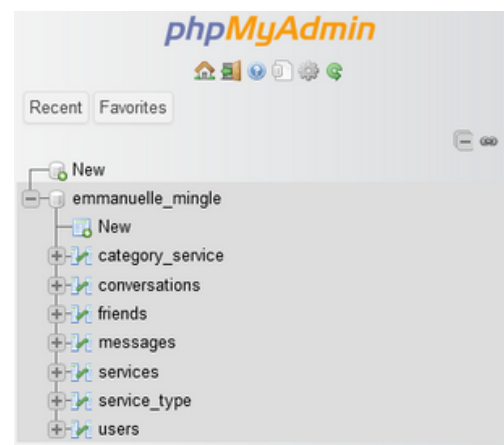
Nom
362570
362570_guillaume
362570_manon
362570_mehdy
362570_renaud

Nous avons ensuite créé toute la base de données et son architecture via l'interface phpMyAdmin proposée par AlwaysData :

```
CREATE DATABASE IF NOT EXISTS `emmanuelle_mingle`  
DEFAULT CHARACTER  
SET utf8mb4 COLLATE utf8mb4_general_ci;  
USE `emmanuelle_mingle`;
```

phpMyAdmin et requêtes SQL

Ce service, souvent utilisé pour administrer MySQL via son interface graphique conviviale, nous a facilité la gestion et l'interaction avec la base de données MySQL.



03 – BASE DE DONNEES

Langage de définition de données

Toujours via l'interface phpMyAdmin nous avons effectué plusieurs requêtes SQL afin de créer l'architecture de la base de données :

Exemples de création d'une table

```
CREATE TABLE `category_service` (  
  `id` int(11) NOT NULL,  
  `titre_categorie` varchar(100) NOT NULL,  
  `titre_sous_categorie` varchar(100) DEFAULT NULL,  
  `category_description` text DEFAULT NULL,  
  `category_image` varchar(255) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE `users` (  
  `id` int(11) NOT NULL,  
  `username` varchar(255) NOT NULL,  
  `mail` varchar(255) NOT NULL,  
  `user_pass` varchar(255) NOT NULL,  
  `localisation` varchar(255) NOT NULL,  
  `avatar` varchar(255) DEFAULT NULL,  
  `biographie` text DEFAULT NULL,  
  `service_count` int(11) DEFAULT NULL,  
  `service_type` varchar(255) DEFAULT NULL,  
  `role` enum('admin','user') NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Une fois nos tables créées, nous les avons peuplés de données fictives afin de pouvoir effectuer nos différents tests

Exemple d'insertion de données dans une table

```
INSERT INTO `category_service` (`id`, `titre_categorie`, `titre_sous_categorie`,  
  `category_description`, `category_image`) VALUES  
(1, 'categorie1', NULL, NULL, NULL),  
(2, 'categorie2', NULL, NULL, NULL),  
(3, 'categorie3', NULL, NULL, NULL),  
(4, 'categorie4', NULL, NULL, NULL),  
(5, 'categorie5', NULL, NULL, NULL),  
(6, 'categorie6', NULL, NULL, NULL),  
(7, 'categorie7', NULL, NULL, NULL),  
(8, 'categorie8', NULL, NULL, NULL),  
(9, 'categorie9', NULL, NULL, NULL),
```

Il nous est arrivé, une fois la phase de développement débutée, de devoir modifier certains éléments des tables (contraintes incorrectes, ajout de clés ...). Voici quelques requêtes SQL illustrant ces modifications :

```
ALTER TABLE `users`  
  ADD PRIMARY KEY (`id`);  
  
ALTER TABLE `category_service`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=45;  
  
ALTER TABLE `conversations`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=220;
```


04 – DÉVELOPPEMENT

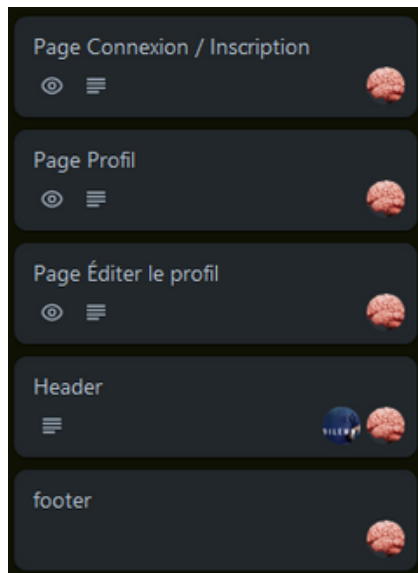
Structure du projet

Un fois toutes les étapes précédentes réalisées, nous avons attaqué la partie développement. Nous avons choisi de travailler avec le framework React pour la partie front, et Express.js (le framework web minimaliste pour Node.js) pour le côté serveur, le tout sous une architecture MVC (Model-View-Controller).

Nous avons mis en place une documentation README afin de fournir les informations essentielles de notre application (Introduction et Contexte, Installation, Utilisation ...)

Répartition

Nous nous sommes réparti le développement des différentes fonctionnalités de l'application. Voici les principales features dont j'ai eu la responsabilité :



Pour plus de cohérence, j'ai aussi la charge de la gestion du backend sur la partie "user "

MVC

Le MVC est un pattern d'architecture logicielle qui sépare les préoccupations de l'application en trois composants distincts : le Modèle, la Vue et le Contrôleur.

Cette séparation nous a permis une meilleure organisation du code, une maintenance et une évolutivité facilitées.

Le **Modèle** a pour fonction de :

- Gérer les données de l'application.
- Effectuer des opérations CRUD (Create, Read, Update, Delete) sur les données.
- Appliquer la logique métier.
- Notifier la Vue des changements de données.

04 – DÉVELOPPEMENT

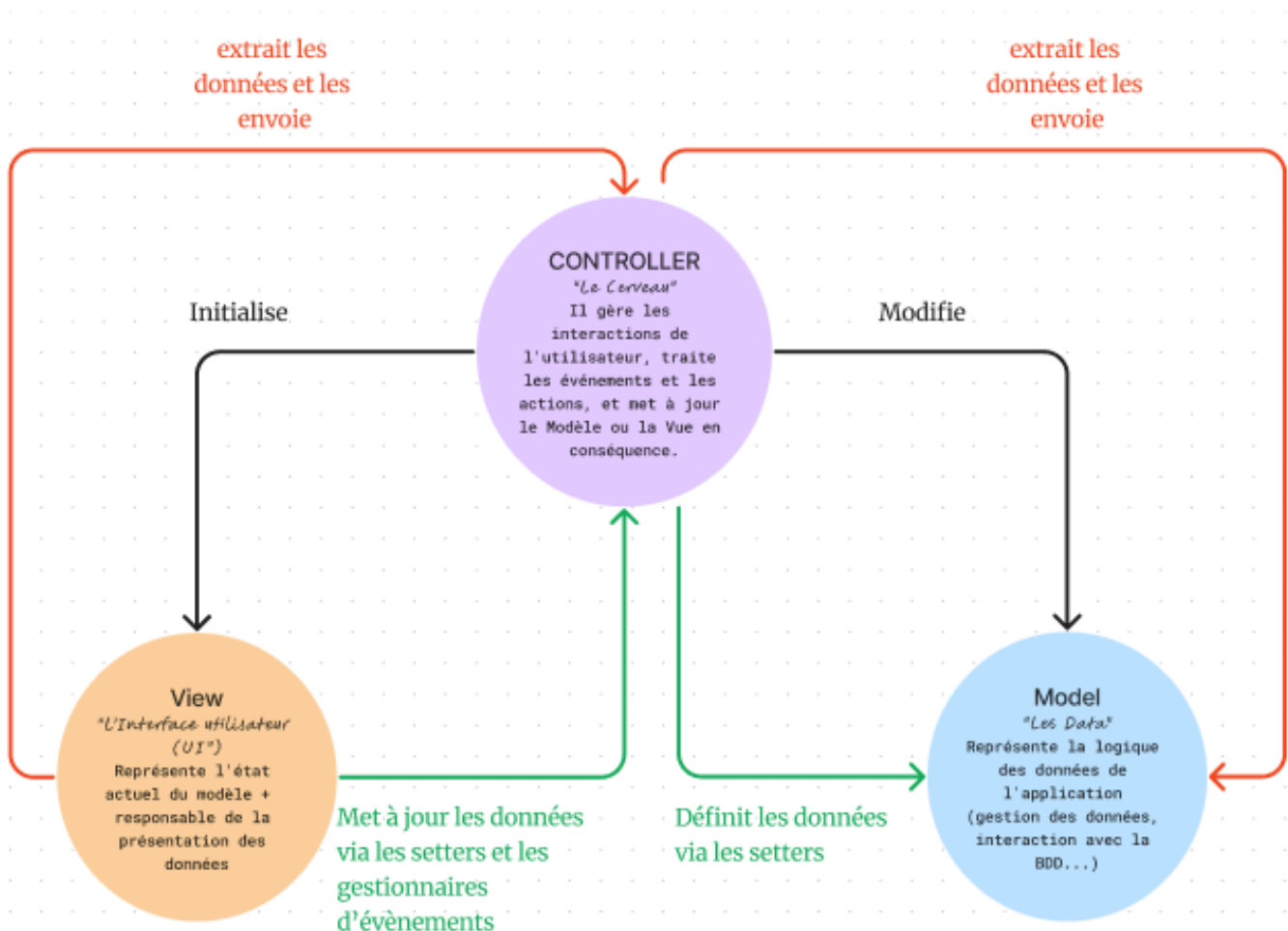
Structure du projet

La **Vue** a pour fonction de :

- Afficher les données fournies par le Modèle.
- Envoyer les interactions de l'utilisateur au Contrôleur.
- Mettre à jour l'interface utilisateur en fonction des changements de données.

Enfin, le **Contrôleur** a pour fonction de :

- Recevoir et traiter les entrées de l'utilisateur.
- Appeler les méthodes appropriées du Modèle pour manipuler les données.
- Sélectionner la Vue appropriée pour afficher les résultats à l'utilisateur.
- Coordonner l'interaction entre le Modèle et la Vue.



Retrouvez notre MVC complet dans la partie "**backend**", p.41

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le frontend a été développé en utilisant le framework Javascript **React**.

C'est une bibliothèque JavaScript open source utilisée pour construire des interfaces utilisateur, notamment pour les applications web à une seule page (Single Page Applications).

Nous avons choisi cette approche afin de mettre jour dynamiquement le contenu sans recharger la page entière, offrant une expérience utilisateur plus fluide.

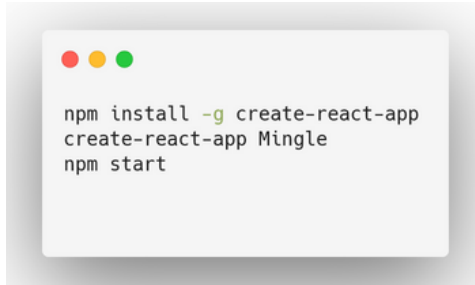
React nous a permis de créer des composants réutilisables qui gèrent leur propre état, facilitant ainsi le développement et la maintenance de l'interface utilisateur.

Installation

Afin d'initialiser notre projet, nous nous sommes servis d'un template téléchargé sur GitHub (voir annexes).

Suivi de la commande suivante afin d'installer toutes les dépendances : **npm install**

Si nous n'avions pas utilisé de template, nous aurions du exécuter les commandes suivantes :



```
npm install -g create-react-app
create-react-app Mingle
npm start
```

Enfin, nous avons utilisé le tool Nodemon, qui redémarre automatiquement le serveur lorsque des changements sont détectés



```
npm install -g nodemon
npm run dev
```

Concernant le CSS, nous avons décidé d'utiliser la librairie **Tailwind**. Cela nous a permis de créer des interfaces utilisateur rapidement et efficacement, grâce à sa collection de classes utilitaires. Nous avons pu les combiner pour construire des interfaces personnalisées, ainsi que créer nos propres thème afin de reprendre la charte graphique définie lors de la phase de pré production.

Une fois l'installation faite, nous avons configuré le fichier `tailwind.config.js` afin d'importer Tailwind dans tous nos fichiers, et personnaliser notre CSS (voir annexes p.58)

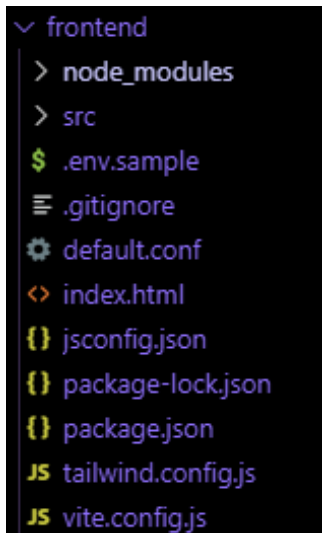
Une fois toutes ces configurations effectuées, nous avons pu partir sur la création de nos composants, cœur du développement avec React. Je vais vous développer les différentes fonctionnalités dont j'ai eu la responsabilité. Pour rappel, cela concerne :

- L'interface de connexion/inscription,
- Les profils utilisateurs (publiques et privés), tant l'interface de consultation que l'interface d'édition,
- Le header (en collaboration avec Mehdy Bouzid),
- Et le footer

04 – DÉVELOPPEMENT

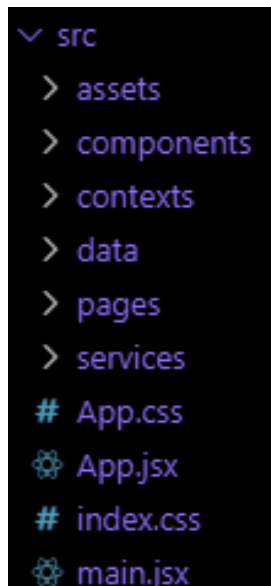
Frontend – Simulateur de “La Vue”

Avant de vous détailler les composants que dont j’ai eu la responsabilité, voici la structure de notre code pour la partie frontend :



- “**node_modules**” : contient toutes les dépendances du projet comme les bibliothèques, les outils de développement et les plugins installés via npm (Node Package Manager)
- “**package-lock.json**” : verrouille les versions des dépendances et assurent que les mêmes versions sont installées à chaque npm install
- “**.gitignore**” : fichier texte utilisé par Git pour déterminer quels fichiers et répertoires doivent être ignorés dans un dépôt (générés automatiquement, qui contiennent des informations sensibles...)
- “**index.html**” : sert de point de départ pour l'application. Il fournit la structure de base du document HTML, définit le point d'entrée pour l'application React, et inclut les scripts et les styles nécessaires => utilisé par les outils de compilation et de build pour générer le fichier HTML final qui sera servi aux utilisateurs.

Dans le fichier “src” nous retrouvons la construction du rendu REACT



- “**components**” et “**pages**” : Nous retrouverons ici tous les composants, qui permettent de construire des interfaces utilisateur de manière modulaire, réutilisable et maintenable
- “**contexts**” : Contient certains contextes, ce qui nous a permis de partager des données à travers l'arborescence des composants sans avoir à passer explicitement des props à chaque niveau. Pour ma part ça concernait les informations de session.
- “**App.jsx**” : composant racine de l'application => assemble et orchestre tous les autres composants.
- “**main.jsx**” : sert de point d'entrée principal pour le rendu de l'application => responsable de l'initialisation de l'application React et de son montage sur un élément du DOM dans le “index.html”

Maintenant la structure étudiée, rentrons dans le vif du sujet avec les composants organisant les rendus dont j’ai eu la charge

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le HEADER

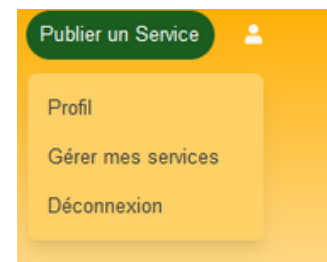


Le Header est un composant à part entière qui a été instancié à la racine de notre projet pour qu’il se retrouve dans toute la navigation de notre site



Il aide les utilisateurs à comprendre rapidement où ils se trouvent et ce qu'ils peuvent faire sur la page. Il contient aussi notre identité avec le logo Mingle.

Enfin, la petite icône à côté de “Publier un Service” permet d’ouvrir un togglemenu menant directement à certains éléments de personnalisation liés au profil.



Afin de construire ce composant, j’ai d’abord dû effectuer tous les imports nécessaires : les bibliothèques d’icônes, les composants reliés (logout ...), ainsi que toutes les méthodes natives de React dont j’aurai besoin (contexte, state, effect ...)



Un fois fait, j’ai dû créer toute la fonction Header. En effet, le fichier .jsx est décomposé en 2 parties :

- La définition du composant React en tant que fonction
- Le “return”, qui est une syntaxe qui ressemble à du HTML mais qui est en fait transformée en JavaScript avant d’être interprétée par le navigateur.

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le HEADER

Définition de la fonction “Header” :

- J’ai effectué plusieurs déclaration d’états(**useState**), permettant de gérer l’ouverture et la fermeture du menu principal, du menu utilisateur et d’extraire les valeurs **isLoggedIn** et **userId** du contexte d’authentification (voir p.22.)
- Ensuite, j’ai utilisé le hook **useEffect** afin de surveiller les changements de **isLoggedIn**.
- Enfin, j’ai déclaré une fonction permettant de gérer les cliques sur les liens du menu

```
const Header = () => {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const [isUserMenuOpen, setIsUserMenuOpen] = useState(false);
  const { isLoggedIn, userId } = useContext(AuthContext);
  const toggleMenu = () => {
    setIsMenuOpen(!isMenuOpen);
  };

  const toggleUserMenu = () => {
    setIsUserMenuOpen(!isUserMenuOpen);
  };

  const closeMenu = () => {
    setIsMenuOpen(false);
  };

  useEffect(() => {
    console.log('isLoggedIn changed:', isLoggedIn);
  }, [isLoggedIn]);

  const handleLinkClick = () => {
    closeMenu();
    setIsUserMenuOpen(false);
  };
};
```

Le rendu de la navbar contient 2 éléments : Les liens directs, et le togglemenu

Navbar

```
<nav className="hidden md:flex md:ml-auto md:space-x-4
items-center relative">
  <Link to="/" className="text-black hover:text-
gray-700">Accueil</Link>
  <Link to="/listeService" className="text-black
hover:text-gray-700">Catégories de services</Link>
  <Link to="/publier" className="text-white hover:text-
gray-700 rounded-full h-10 bg-green-900 flex items-center
justify-center px-3">Publier un Service</Link>
```

togglemenu

```
<div className="relative">
  <FaUser
    className="text-white ml-2 cursor-pointer"
    onClick={toggleUserMenu}
  />
  {isUserMenuOpen && (
    <div className="absolute right-0 w-48 bg-secondary
rounded-lg shadow-lg py-2 mt-6">
      {isLoggedIn ? (
        <Link
          to={`/profil/${userId}`}
          className="block px-4 py-2 text-gray-800
hover:bg-yellow-400"
          onClick={handleLinkClick}
        >
          Profil
        </Link>

        <Link
          to="/dashboard"
          className="block px-4 py-2 text-gray-800
hover:bg-yellow-400"
          onClick={handleLinkClick}
        >
          Gérer mes services
        </Link>
        <Logout closeMenu={closeMenu} />
      ) : (
        <Link
          to="/ConnexionInscription"
          className="block px-4 py-2 text-gray-800
hover:bg-yellow-400"
          onClick={handleLinkClick}
        >
          Connexion/Inscription
        </Link>
      )}
    </div>
  )}
</div>
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le HEADER

Ces rendus sont personnalisés grâce aux classes utilitaires de Tailwind. Par exemple :

```
className="block px-4 py-2 text-gray-800 hover:bg-yellow-400"
```

- **block** : définit l'affichage de l'élément comme `display: block`
- **px-4** : ajoute un padding horizontal de 4 unités à l'élément, c'est à dire 1rem (16px)
- **py-2** : ajoute un padding vertical de 2 unités à l'élément, c'est à dire 0.5rem (8px)
- **text-gray-800** : classe de couleur de texte dans Tailwind CSS. `text-gray` suivi du numéro indique une nuance spécifique de gris.
- **hover:bg-yellow-400** : classe de couleur de fond au survol dans Tailwind CSS. `hover:bg` suivi de la couleur et du numéro indique une nuance spécifique de couleur jaune.

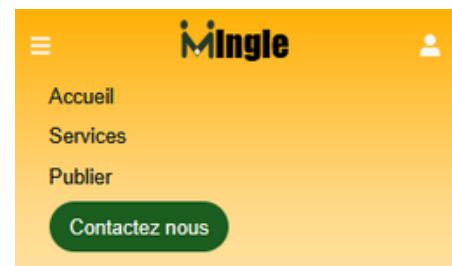
Ces classe utilitaires m'ont permis d'appliquer un css harmonieux rapidement, et surtout responsive.

Dans ce rendu, j'ai aussi appliqué un affichage conditionnel suivant l'état que connexion de l'utilisateur (données récupérées dans les fonctions définies au préalable). Lorsque l'utilisateur n'est pas connecté, seul le lien vers la page de connexion/inscription est disponible. S'il est connecté, il aura accès à toutes les option du menu utilisateur. (voir annexes)

Enfin, pour améliorer l'accessibilité et l'expérience utilisateur, j'ai crée une condition particulière pour l'affichage en version mobile : la navbar se stock dans un menu burger, qui propose les mêmes options lorsqu'on clique dessus

```
{isMenuOpen && (
  <div className="md:hidden flex flex-col items-start pl-4
mt-2 space-y-2">
    <Link to="/" className="text-black hover:text-gray-700"
onClick={handleLinkClick}>Accueil</Link>
    <Link to="/service" className="text-black hover:text-
gray-700" onClick={handleLinkClick}>Services</Link>
    <Link to="/publier" className="text-black hover:text-
gray-700" onClick={handleLinkClick}>Publier</Link>
    <Link to="/formulaire" className="text-white hover:text-
gray-700 rounded-full h-10 bg-green-900 flex items-center
justify-center px-4" onClick={handleLinkClick}>Contactez nous
</Link>
```

Rendu du Header en version mobile



04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le FOOTER

[À propos](#) [FAQ](#) [Conditions d'utilisation](#) [Politique de confidentialité](#) [Contact](#)

© 2024 Mingle. Tous droits réservés.

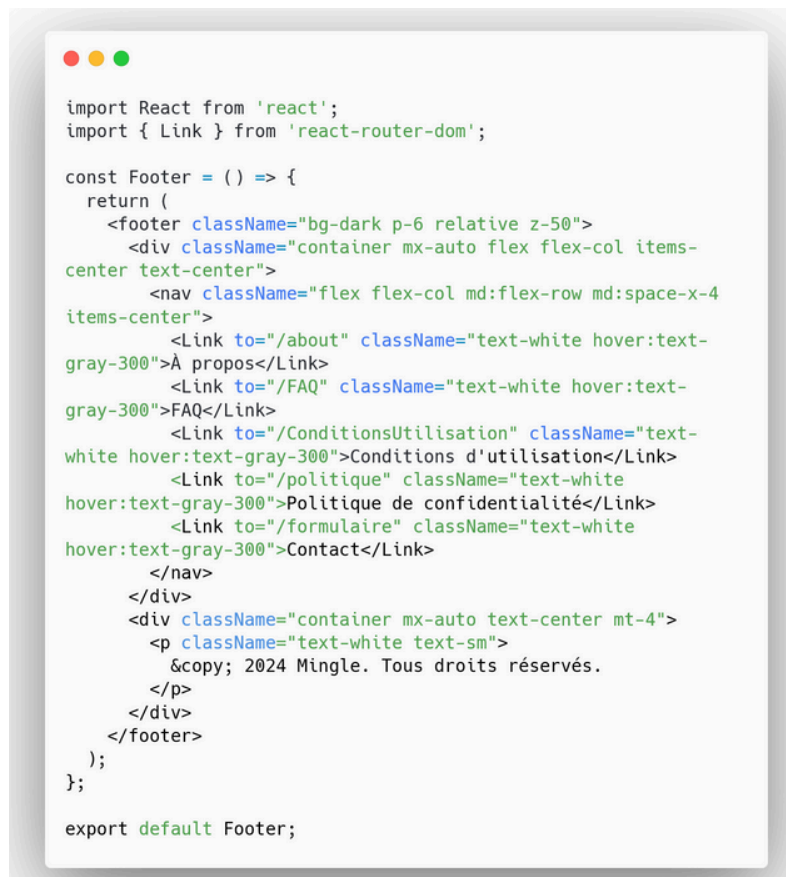
Le Footer a le même comportement que le Header, à savoir qu'il doit être instancié à la racine du projet pour se retrouver sur toutes les pages, et qu'il doit être responsive.



Rendu du Footer en version mobile



Le Footer a un code beaucoup plus simple. Il n'y a pas de fonctions à définir, juste un rendu jsx, comprenant les liens (via router-dom) redirigeant vers les pages, et la personnalisation via Tailwind (css et responsive)



04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le FOOTER

Explication du css de la div principale :

- “**container**” : Définit un conteneur centré horizontalement avec des marges automatiques sur les côtés (mx-auto) => contenu à l'intérieur du conteneur sera centré horizontalement sur la page.
- “**flex**” : Transforme le conteneur en un conteneur flex => manipuler facilement la disposition des éléments enfants.
- “**flex-col**” : Définit la direction des éléments enfants du conteneur flex comme étant en colonne (de haut en bas).
- “**items-center**” et “**text-center**” : Les éléments enfants seront centrés horizontalement sur l'axe principal du conteneur (axe Y si flex-col est utilisé) et centre le texte à l'intérieur de l'élément.

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Avant de vous détailler le développement de la partie “Profil”, “Connexion” et “Inscription” je dois d’abord vous détailler mon contexte d’authentification AuthContext, qui a beaucoup été utilisé (comme dans le header)

Le AUTHCONTEXT

Le contexte d’authentification m’a permis de maintenir et de partager de manière centralisée l’état d’authentification (connexion/déconnexion) à travers toute notre application. Cela simplifie la gestion de l’authentification et améliore la réactivité de l’interface utilisateur.

J’ai pu gérer l’état d’authentification centralisé et permettre à différents composants d’accéder et de modifier cet état sans avoir besoin de passer des données via des props à travers plusieurs niveaux de composants.

J’ai tout d’abord importé les modules nécessaires (React, createContext, useState, useEffect, axios).

J’ai ensuite créé un nouveau contexte d’authentification (AuthContext) en utilisant “createContext()”. Ce contexte sera utilisé pour partager l’état d’authentification à travers toute l’application :

```
import React, { createContext, useState, useEffect } from
'react';
import axios from 'axios';

export const AuthContext = createContext();
```

J’ai ensuite utilisé “AuthProvider” (composant React) qui enveloppe ses enfants avec le contexte d’authentification.

Il utilise “useState” pour déclarer trois états : “isLoggedIn” (statut de connexion de l’utilisateur), “userId” (identifiant de l’utilisateur connecté) et “token” (jeton d’authentification).

Le “useEffect” est quant à lui utilisé pour charger l’état initial à partir du localStorage. Cela permet de maintenir la connexion de l’utilisateur même après le rafraîchissement de la page.

```
export const AuthProvider = ({ children }) => {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [userId, setUserId] = useState(null);
  const [token, setToken] = useState(null);

  useEffect(() => {
    const token = localStorage.getItem('token');
    const userId = localStorage.getItem('userId');
    setIsLoggedIn(!!token && !!userId);
    setUserId(userId);
    setToken(token);
  }, []);

  return (
    <AuthContext.Provider value={{
      isLoggedIn,
      userId,
      token,
      setIsLoggedIn,
      setUserId,
      setToken
    }}>
      {children}
    </AuthContext.Provider>
  );
};
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Le AUTHCONTEXT

Puis j'ai déclaré une fonction asynchrone “login” qui envoie une requête POST au serveur pour authentifier l'utilisateur avec un email et un mot de passe.

- Si la connexion réussit (statut 201), elle stocke le token et l'userId dans le localStorage et met à jour les états isLoggedIn, userId et token.
- En cas d'échec, elle affiche une erreur.

```
const login = async (email, password) => {
  try {
    const response = await axios.post('http://localhost:5000/
user/login', {
      mail: email,
      user_pass: password,
    });

    if (response.status === 201) {
      const { token, userId } = response.data;
      localStorage.setItem('token', token);
      localStorage.setItem('userId', userId);
      setIsLoggedIn(true);
      setUserId(userId);
      setToken(token);
    } else {
      console.error('Login failed:', response.status);
    }
  } catch (error) {
    console.error('Error logging in:', error);
  }
};
```

La fonction “logout” est une fonction qui supprime le token et le userId du localStorage, et met à jour les états pour indiquer que l'utilisateur n'est plus connecté.

```
const logout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('userId');
  setIsLoggedIn(false);
  setUserId(null);
  setToken(null);
};
```

Enfin, “AuthProvider” rend les valeurs isLoggedIn, userId, login, logout et token disponibles pour tous les composants enfants enveloppés par ce contexte.

Cela permet à n'importe quel composant de l'application d'accéder à l'état d'authentification actuel et aux fonctions pour se connecter et se déconnecter.

```
return (
  <AuthContext.Provider value={{ isLoggedIn, userId, login,
logout, token }}>
    {children}
  </AuthContext.Provider>
);
```

04 – DÉVELOPPEMENT

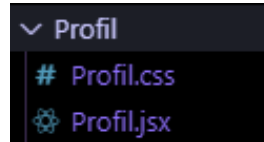
Frontend – Simulateur de “La Vue”

Maintenant le contexte d'authentification posé, nous allons développer toute la partie concernant les profils utilisateurs (voir le rendu en annexes)

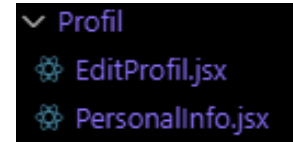
Les PROFILS

Plusieurs composants ont été créés pour fabriquer ce rendu.

Page parente



Pages enfants



Les PROFILS - Parent “Profil.jsx”

Commençons par la page parente **Profil.jsx**

Comme pour les autres composants j'ai commencé par tous mes imports.

- “**PersonnalInfo**” et “**ServiceCard**” sont des composants enfants.
- “**react-toastify**” est une librairie React qui m'a servi à afficher des popup pour améliorer l'expérience utilisateur
- “**Axios**” est une bibliothèque JavaScript qui m'a servi pour l'envoi de requêtes asynchrones vers notre API REST



J'ai ensuite procédé à la définition de ma fonction “**Profil**” :

J'ai déclaré les états via “**useState**” (userData, isDeleted, isLoggedIn, services) et les fonctions pour les mettre à jour.

J'ai utilisé “**useParams**” pour récupérer les paramètres d'URL, ici userId.

Je me suis servi du **localStorage** récupérer l'ID de l'utilisateur actuellement connecté.

“**useNavigate**” m'a permis de naviguer de manière impérative vers une autre page.



04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Parent “Profil.jsx”

J’ai ensuite déterminé une fonction “**fetchData**” pour récupérer les données utilisateur :

“**fetchData**” est une fonction asynchrone qui utilise axios pour récupérer les données utilisateur (userResponse) et les services associés (servicesResponse).

Je me suis servi de la méthode “**localStorage.getItem("token")**” pour récupérer le token d'authentification dans le local storage.

“**axios.get**” effectue des requêtes GET vers notre API backend pour récupérer les données utilisateur et services.

Et enfin **setUserData** et **setServices** met à jour les états userData et services avec les données reçues.

```
const fetchData = async () => {
  try {
    const token = localStorage.getItem("token");
    const headers = token ? { Authorization: `Bearer ${token}` }
    : {};
    const userResponse = await axios.get(
      `http://localhost:5000/users/${userId}`,
      { headers }
    );
    setUserData(userResponse.data);
    if (token) {
      setIsLoggedIn(true);
    }
    const servicesResponse = await axios.get(
      `http://localhost:5000/service?userId=${userId}`,
      { headers }
    );
    setServices(servicesResponse.data.slice(0, 3));
  } catch (error) {
    console.error("Error fetching data", error);
  }
};
```

Je me suis ensuite attaquée à la fonctionnalité de suppression du profil utilisateur via la fonction asynchrone “**handleDeleteProfile**”.

“**axios.delete**” se charge d’envoyer une requête DELETE vers notre API backend pour supprimer le profil.

La suppression du token et de l’ID utilisateur passe par la méthode “**localStorage.removeItem**” après la suppression du profil.

Enfin l’utilisateur reçoit une notification (“**toast.success**”) issue de “react-toastify” pour en cas de succès après la suppression.

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Parent “Profil.jsx”

```
const handleDeleteProfile = async () => {
  try {
    const token = localStorage.getItem("token");

    const response = await axios.delete(
      `http://localhost:5000/users/${userId}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      }
    );

    if (response.status === 204) {
      console.log("Profil supprimé avec succès.");
      localStorage.removeItem("userId");
      localStorage.removeItem("token");
      toast.success("Votre profil a été supprimé avec succès.", {
        position: "top-center",
        className: "custom-toast",
        autoClose: 2000,
      });
      setIsDeleted(true);
    } else {
      console.error("Erreur lors de la suppression du profil");
    }
  } catch (error) {
    console.error("Erreur lors de la suppression du profil", error);
  }
};
```

Enfin, pour clôturer la fonction “Profil”, j’ai utilisé un “useEffect” pour appeler “fetchData” une seule fois après le 1er rendu et récupérer les données utilisateur.

J’ai utilisé un 2e “useEffect(isDeleted)” qui surveille l’état “isDeleted” pour rediriger l’utilisateur vers la page d’accueil (“/”) après 3 secondes si le profil a été supprimé.

```
useEffect(() => {
  fetchData();
}, []);

useEffect(() => {
  if (isDeleted) {
    setTimeout(() => {
      window.location.href = "/";
    }, 3000);
  }
}, [isDeleted]);
```

Une fois ma fonction “Profil” montée, je me suis attaqué au rendu du composant, qui va afficher le contenu du profil utilisateur avec des informations personnelles (composant “PersonalInfo”) et les 3 derniers services publiés (composant “Servicecard”).

La condition “!userData” va afficher un message de chargement tant que les données utilisateur ne sont pas chargées.

La gestion de l’état d’authentification “isLoggedIn” va afficher différents éléments en fonction de l’état de connexion de l’utilisateur (le bouton “éditer” par exemple)

La gestion des événements “handleSeeMoreClick” va rediriger l’utilisateur vers son tableau de bord (“/dashboard”) lorsqu’il clique sur “Voir plus” et qu’il est connecté.

“isCurrentUser” permet de vérifier si l’utilisateur connecté est le possesseur de lu profil. Si ce n’est pas le cas, le profil est alors disponible en simple consultation.

Enfin, “userData.username” permet de personnaliser le titre de la page avec l’id de l’utilisateur.

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Parent “Profil.jsx”

```
return (
  <div className="container mx-auto px-4 py-8">
    <h1 className="text-3xl font-bold mb-6 text-center text-darkslategray">
      Profil Mingle de {userData.username}
    </h1>
    <PersonalInfo
      userData={userData}
      isLoggedIn={isLoggedIn}
      isCurrentUser={isCurrentUser}
      onDelete={handleDeleteProfile}
    />
    <div className="mt-8 bg-white rounded-lg border border-gray-300 shadow-2xl p-4">
      <h2 className="text-2xl font-extrabold mb-6 text-center text-darkslategray">
        Services publiés
      </h2>
      <div className="flex flex-wrap gap-3">
        {services.map((service) => (
          <Servicecard key={service.id} service={service} />
        ))}
      </div>
      <div className="text-center mt-4">
        {isLoggedIn ? (
          <button
            className="bg-blue-600 hover:bg-blue-700 text-white py-2 px-4 rounded-lg text-lg font-medium focus:outline-none focus:shadow-outline"
            onClick={handleSeeMoreClick}
          >
            Voir plus
          </button>
        ) : (
          <p className="text-gray-500">
            Connectez-vous pour voir plus de détails
          </p>
        )}
      </div>
    </div>
    <ToastContainer />
  </div>
);
```

Les PROFILS - Enfant “PersonalInfo.jsx”

Une fois le composant parent “Profil” opérationnel, je me suis attaquée au bloc d’informations personnelles, avec le composant “PersonalInfo”.

Comme pour les autres composants, j’ai effectué mes imports (ici “React” et “useNavigate”), et j’ai commencé la construction de la fonction “PersonalInfo”. Cette fonction prends en props `userData`, `isLoggedIn`, `isCurrentUser`, et `onDelete`.

Le “`useNavigate()`” initialise le hook `useNavigate` pour permettre la navigation.

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

```
import React from 'react';
import { useNavigate } from 'react-router-dom';

const PersonalInfo = ({ userData, isLoggedIn, isCurrentUser,
  onDelete }) => {
  const navigate = useNavigate();
```

J’ai ensuite créé une fonction “**handleEditClick**” qui navigue vers la page d’édition du profil avec les données de l’utilisateur lorsque le bouton d’édition est cliqué.

```
const handleEditClick = () => {
  navigate('/EditProfil', { state: { userData } });
};
```

Enfin, pour terminer ma fonction “**Profil**”, j’ai créé une fonction “**handleDeleteClick**” qui affiche une fenêtre de confirmation avant de déclencher la fonction “**onDelete**” si l’utilisateur confirme la suppression du profil.

```
const handleDeleteClick = () => {
  if (window.confirm("Êtes-vous sûr de vouloir supprimer votre
    profil ? Cette action est irréversible.")) {
    onDelete();
  }
};
```

Une fois fait, j’ai développé le rendu jsx.

Ce premier bloc définit les paires d’étiquettes et de paragraphes pour afficher les différentes propriétés de l’utilisateur (nom, email, ville, type de service proposé, photo de profil biographie). Ils sont stylisés via les classes utilitaires **Tailwind**, tant pour le style que pour le responsive (**flex**, **md:flex-row** ...)

Exemple avec les labels “Nom” et “Email”

```
return (
  <div className="w-full p-8 border-solid bg-white rounded-lg
    border border-gray-300 shadow-2xl">
    <h2 className="text-2xl font-extrabold mb-6 text-center
      text-darkslategray">Informations personnelles</h2>
    <div className="flex flex-col md:flex-row gap-8 p-4">
      <div className="flex-1 space-y-4">
        <div>
          <label className="block text-darkslategray">Nom</
        label>
        <p className="mt-1 text-m rounded-lg border border-
          darkslategray p-1">{userData.username}</p>
        </div>
        <div>
          <label className="block text-darkslategray">Email</
        label>
        <p className="mt-1 text-m rounded-lg border border-
          darkslategray p-1">{userData.mail}</p>
        </div>
      </div>
    </div>
  </div>
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Enfant “PersonalInfo.jsx”

Enfin `{isLoggedIn && isCurrentUser && (...);}` vérifie si l'utilisateur est connecté (`isLoggedIn`) et s'il s'agit du profil de l'utilisateur actuel (`isCurrentUser`). Si c'est le cas, affiche les boutons d'édition et de suppression.

Le bouton “Editer” appelle “`handleEditClick`” lorsqu'il est cliqué. Le bouton “Supprimer” appelle “`handleDeleteClick`” lorsqu'il est cliqué.

```
{isLoggedIn && isCurrentUser && (
  <div className="mt-4 text-center space-x-4 flex">
    <button className="bg-accent hover:bg-blue-700 text-
white py-2 px-9 rounded-lg text-lg font-medium focus:outline-
none focus:shadow-outline" onClick={handleEditClick}>
      Éditer
    </button>
    <button className="bg-red-600 hover:bg-red-700 text-
white py-2 px-7 rounded-lg text-lg font-medium focus:outline-
none focus:shadow-outline" onClick={handleDeleteClick}>
      Supprimer
    </button>
  </div>
)}
</div>
);
};
```

Les PROFILS - Enfant “EditProfil.jsx”

Passons maintenant au dernier composant enfant du profil, “`EditProfil`”

J'ai de nouveau importer les bibliothèques nécessaires : `React` pour les composants, `Formik` pour la gestion des formulaires, `Yup` pour la validation, `react-router-dom` pour la navigation, et `axios` pour les requêtes HTTP.

```
import React from "react";
import { Formik, Form, useField } from 'formik';
import * as Yup from 'yup';
import { useNavigate, useLocation } from 'react-router-dom';
import axios from "axios";
```

Ensuite, j'ai déclaré un schéma de validation “`Yup`” pour définir et sécuriser côté frontend les champs du formulaire.

```
const validationSchema = Yup.object({
  username: Yup.string().required('Champs obligatoire'),
  mail: Yup.string().email('Adresse mail
invalide').required('Champs obligatoire'),
  localisation: Yup.string().required('Champs obligatoire'),
  biographie: Yup.string(),
  service_type: Yup.string().required('Champs obligatoire'),
  avatar: Yup.mixed(),
});
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Enfant “EditProfil.jsx”

Ensuite, j’ai déclaré la fonction “**MyTextInput**”, qui rend un champ de saisie avec une étiquette et affiche des erreurs de validation le cas échéant. Il accepte des propriétés (props) et les déstructure pour obtenir label et les autres props.

Il utilise aussi le hook “**useField**” de Formik pour lier le champ au formulaire avec “**field**”, tandis que “**meta**” contient des informations d’état sur le champ (touched, indiquant si le champ a été touché ou error, message d’erreur de validation si le champ a une erreur.

```
const MyTextInput = ({ label, ...props }) => {
  const [field, meta] = useField(props);
  return (
    <div className="mb-4">
      <label className="block text-darkslategray text-sm font-
bold mb-2" htmlFor={props.id || props.name}>{label}</label>
      <input
        className="mt-1 text-m rounded-lg border border-
darkslategray p-1 w-full max-w-lg"
        {...field} {...props}
      />
      {meta.touched && meta.error ? (
        <div className="text-red-500 text-xs
italic">{meta.error}</div>
      ) : null}
    </div>
  );
};
```

La fonction “**MyTextArea**”, rend une zone de texte avec une étiquette et affiche des erreurs de validation si nécessaire.

Il est construit sous le même modèle que “**MyTextInput**”

```
const MyTextarea = ({ label, ...props }) => {
  const [field, meta] = useField(props);
  return (
    <div className="mb-4 mx-auto flex justify-center">
      <label className="block text-darkslategray text-sm font-
bold mb-2" htmlFor={props.id || props.name}>{label}</label>
      <textarea
        className="mt-1 text-m rounded-lg border border-
darkslategray p-1 w-full max-w-lg"
        {...field} {...props}
      />
      {meta.touched && meta.error ? (
        <div className="text-red-500 text-xs
italic">{meta.error}</div>
      ) : null}
    </div>
  );
};
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Enfant “EditProfil.jsx”

La fonction “MyFileInputInput” gère les champs de saisie de fichiers et affiche l'aperçu du fichier sélectionné.

La gestion des changements utilise “FileReader” pour lire les fichiers en tant qu'URL de données et les affecter au champ. Le reste de la fonction reprend le schéma des deux fonctions ci-dessus

```
const MyFileInput = ({ label, ...props }) => {
  const [field, meta, helpers] = useField(props);
  const { setValue } = helpers;

  const handleChange = (e) => {
    const file = e.currentTarget.files[0];
    const reader = new FileReader();
    reader.onloadend = () => {
      setValue(reader.result);
    };
    if (file) {
      reader.readAsDataURL(file);
    }
  };

  return (
    <div className="mb-4">
      <label className="block text-darkslategray text-sm font-
bold mb-2" htmlFor={props.id || props.name}>{label}</label>
      <input
        className="shadow appearance-none border rounded w-full
py-2 px-3 text-darkslategray leading-tight focus:outline-none
focus:shadow-outline"
        type="file"
        onChange={handleChange}
      />
      {meta.touched && meta.error ? (
        <div className="text-red-500 text-xs
italic">{meta.error}</div>
      ) : null}
      {field.value && (
        <div className="text-sm mt-2">{typeof field.value ===
'string' ? 'Image sélectionnée' : field.value.name}</div>
      )}
    </div>
  );
};
```

Il est tant maintenant d'aborder la fonction principale, “EditProfil”.

Elle utilise “useLocation” et “useNavigate” pour récupérer les données de l'utilisateur depuis l'état de la navigation et initialiser la navigation.

On retrouve aussi une définition des valeurs initiales du formulaire à partir des données de l'utilisateur.

```
const EditProfil = () => {
  const location = useLocation();
  const navigate = useNavigate();
  const { userData } = location.state || { userData: {} };

  const initialValues = {
    username: userData.username || '',
    mail: userData.mail || '',
    localisation: userData.localisation || '',
    biographie: userData.biographie || '',
    service_type: userData.service_type || '',
    avatar: userData.avatar || '',
  };
};
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Les PROFILS - Enfant “EditProfil.jsx”

J’ai ensuite déclaré une fonction asynchrone “**handleSubmit**” gérer la soumission du formulaire. Elle envoie les données au serveur via une requête PUT pour mettre à jour le profil en fonction de l’id de l’utilisateur stocké dans le local storage, et affiche les messages d’erreur si la mise à jour échoue.

```
const handleSubmit = async (values) => {
  try {
    const payload = {
      username: values.username,
      mail: values.mail,
      localisation: values.localisation,
      biographie: values.biographie,
      service_type: values.service_type,
      avatar: values.avatar,
    };
    if (values.user_pass) {
      payload.user_pass = values.user_pass;
    }
    const userId = localStorage.getItem('userId');
    const response = await axios.put(`http://localhost:5000/users/${userId}`, payload, {
      headers: {
        'Authorization': `Bearer ${localStorage.getItem('token')}`,
        'Content-Type': 'application/json',
      },
    });
    if (response.status === 204) {
      console.log('Profil mis à jour avec succès.');
```

Exemple avec “MyTextInput”

Enfin, j’ai organisé le rendu du formulaire avec Formik, pour gérer le formulaire et sa validation.

Les champs personnalisés utilise les composants “MyTextInput”, “MyTextarea” et “MyFileInput”, et j’ai rajouté un bouton de soumission pour enregistrer les modifications du profil.

```
return (
  <Formik
    initialValues={initialValues}
    validationSchema={validationSchema}
    onSubmit={handleSubmit}
  >
    {({ setFieldValue, values }) => (
      <Form className="max-w-screen-lg mx-auto px-6 py-8 rounded-lg shadow-lg">
        <h1 className="text-3xl font-bold mb-6 text-center text-darkslategray">Editer le profil</h1>
        <div className="md:flex md:space-x-8 mb-8 bg-white p-4 rounded-lg border border-gray-300 shadow-2xl">
          <div className="md:w-1/2 bg-white rounded-lg p-4">
            <h2 className="text-2xl font-bold mb-4 text-darkslategray text-center">Informations personnelles</h2>
            <MyTextInput label="Nom" name="username" type="text"/> { /* Corrigé de 'name' */ }
            <MyTextInput label="Email" name="mail" type="email"/> { /* Corrigé de 'email' */ }
            <MyTextInput label="Ville" name="localisation" type="text"/> { /* Corrigé de 'ville' */ }
          </div>
        </div>
      </Form>
    )
  }
);
```

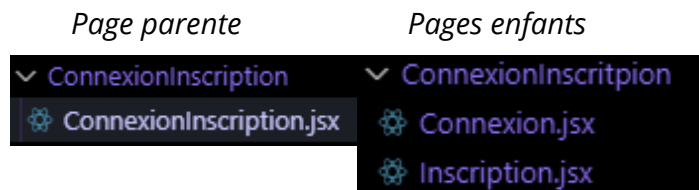

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Connexion/Inscription

Passons à la dernière partie dont j'ai eu la responsabilité : la page de connexion et d'inscription.

Comme pour les profils, il y a une page parente, et 2 pages enfants



Connexion/Inscription - Parents “ConnexionInscription.jsx”

J'ai commencé, comme dans les autres composants, par importer toutes les librairies nécessaires (`useState`, `ToastContainer`, `React` ...), ainsi que les composants enfants.

Ensuite, j'ai déclaré ma fonction “`ConnexionInscription`”, qui va servir écrire une logique qui, en version mobile, affichera 2 boutons, un “Connexion” et un “Inscription”, qui se dérouleront lorsqu'ils seront cliqués (grâce à un “`useState`”, et qui en version desktop, affichera les 2 formulaires côte à côte

```
export default function ConnexionInscription() {
  const [activeComponent, setActiveComponent] = useState("");
  // Gère l'état local d'activeComponent, qui va stocker quelle
  // valeur sera active : soit <Connexion/> soit <Inscription/>
  // 'setActiveComponent' va mettre à jour cet état.

  const renderComponent = () => {
    if (activeComponent === "connexion") {
      return <Connexion />;
    } else if (activeComponent === "inscription") {
      return <Inscription />;
    }
  };
}
```

J'ai terminé par rédigé le rendu de la page (voir annexes).

J'ai créé 2 div, une englobant la logique de l'affichage mobile, et une desktop.

Je me suis servi des classes utilitaires de Tailwind pour le CSS et la mise en forme, ainsi que pour indiquer les breakpoints où l'affichage bascule en version mobile.

Les boutons “Connexion” et “Inscription” mettent à jour `activeComponent` lorsqu'ils sont cliqués, et utilise classes conditionnelles pour styliser le bouton en fonction de l'état actif.

Exemple avec le bouton “Connexion”

```
<button
  className={ `w-full py-2 px-4 rounded
  ${activeComponent === "connexion" ? "bg-accent text-white" :
  "bg-white text-white"} `}
  onClick={() => setActiveComponent("connexion")}
>
  Connexion
</button>
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Connexion/Inscription - Enfant “Connexion”

Passons maintenant au 1er enfant, le composant “Connexion”.

Comme d’habitude on commence avec les imports des librairies (Yup, useFormik, useNavigate et useLocation), ainsi que mon contexte d’Authentification.

J’ai ensuite déclaré ma fonction principale “Connexion”. J’y ai utilisé les hook “navigate” (pour la navigation) et “location” (pour obtenir l’emplacement actuel)

J’ai stocké “login”, qui est une fonction de connexion obtenue à partir du contexte d’authentification.

Et enfin, on retrouve “fromPublier” qui va vérifier si l’utilisateur vient de la page “/Publier.”

```
const navigate = useNavigate();
const location = useLocation();
const { login } = useContext(AuthContext);
const [fromPublier, setFromPublier] = useState(false);
```

Puis j’ai utilisé le hook “useFormik” pour gérer le formulaire, déterminer des valeurs initiales et créer un schéma de validation pour sécuriser le formulaire côté frontend.

```
const formik = useFormik({
  initialValues: {
    email: "",
    password: "",
  },
  validationSchema: Yup.object({
    email: Yup.string()
      .email("Adresse email invalide")
      .required("Champs obligatoire"),
    password: Yup.string().required("Champs obligatoire"),
  }),
});
```

La fonction “onSubmit” est appelée lors de la soumission du formulaire. Elle tente de connecter l’utilisateur et le redirige en fonction d’où il vient : l’accueil ou la page “publier”. Pour rappel, le contexte d’authentification se charge de vérifier si l’utilisateur existe, et de stocker son token et son id dans le local storage.

```
onSubmit: async (values, { setSubmitting }) => {
  try {
    await login(values.email, values.password);
    if (fromPublier) {
      navigate('/Publier');
    } else {
      navigate('/');
    }
  } catch (error) {
    console.error('Error logging in:', error);
  } finally {
    setSubmitting(false);
  }
};
```


04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Connexion/Inscription - Enfant “Connexion”

Enfin, le rendu du composant est géré avec Formik (et Tailwind pour le CSS), et rappelle la fonction “onSubmit”

```
<div className="mb-4">
  <label className="block text-darkslategray"
    htmlFor="login-email">Email</label>
  <input
    className="w-full px-3 py-2 border rounded"
    id="login-email"
    name="email"
    type="email"
    onChange={formik.handleChange}
    onBlur={formik.handleBlur}
    value={formik.values.email}
  />
  {formik.touched.email && formik.errors.email ? (
    <div className="text-red-500 text-sm">{formik.errors.email}</div>
  ) : null}
  ...
  <div>
    <button type="submit" className="w-full bg-blue-500
      text-white py-2 rounded">Se connecter</button>
  </div>
</form>
) : (
  <form onSubmit={formik.handleSubmit}>
```

Connexion/Inscription - Enfant “Inscription”

Terminons avec le dernier enfant, le composant “Inscription”.

J’ai importé une fois de plus les bibliothèques nécessaires (React, useEffect et useState, useFormik, axios pour la requête HTTP POST, Yup et react-toastify).

J’ai déclaré ma fonction “Inscription”.

L’initialisation de Formik est exactement la même que pour le composant “Connexion”, un peu plus poussée car il y a plus de champs. J’ai rendu la sécurisation du formulaire similaire à celle du backend pour plus de cohérence.

```
validationSchema: Yup.object({
  username: Yup.string()
    .required("Nom d'utilisateur obligatoire")
    .min(3, "Le nom d'utilisateur doit comporter au moins 3 caractères"),
  email: Yup.string()
    .email("Adresse email invalide")
    .required("Adresse email obligatoire"),
  password: Yup.string()
    .required("Requis")
    .min(6, "Le mot de passe doit comporter au moins 6 caractères")
    .matches(/!@#%&*(),.?":{}|<>|/, "Le mot de passe doit comporter au moins 1 caractère spécial"),
  confirmPassword: Yup.string()
    .oneOf([
      Yup.ref("password"), null],
    "Les mots de passe doivent correspondre")
    .required("Requis"),
  localisation: Yup.string()
    .required("Localisation obligatoire")
    .min(2, "La ville doit comporter au moins 2 caractères"),
  acceptTerms: Yup.boolean().oneOf([
    true],
    "Accepter les termes et conditions est requis")
}),
}),
```

04 – DÉVELOPPEMENT

Frontend – Simulateur de “La Vue”

Connexion/Inscription - Enfant “Inscription”

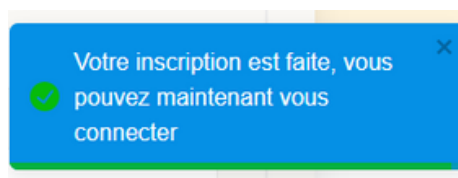
On retrouve ensuite une fonction “onSubmit”, appelée lors de la soumission du formulaire. Elle vérifie si les mots de passe correspondent, récupère les valeurs des champs, envoie une requête POST pour enregistrer l'utilisateur, réinitialise le formulaire et affiche des notifications en cas de succès ou d'échec.

```
onSubmit: async (values, { setSubmitting, resetForm }) => {
  if (values.password !== values.confirmPassword) {
    console.log("Les mots de passe ne correspondent pas");
    return;
  }
  try {
    const response = await axios.post("http://localhost:5000/users", {
      username: values.username,
      mail: values.email,
      user_pass: values.password,
      localisation: values.localisation,
    });
    console.log("User registered successfully:", response.data);
    resetForm();
    toast.success("Votre inscription est faite, vous pouvez maintenant vous connecter", {
      position: "top-center",
      className: "custom-toast",
      autoClose: 2000,
    });
  } catch (error) {
    console.error("Error registering user:", error);
    toast.error("Une erreur s'est produite lors de l'inscription");
  } finally {
    setSubmitting(false);
  }
},
});
```

Enfin, le rendu du composant est encore une fois géré avec Formik. Voici un exemple avec le champs “Ville” :

```
<div className="mb-4">
  <label
    className="block text-darkslategray"
    htmlFor="signup-localisation"
  >
    Ville
  </label>
  <input
    className="w-full px-3 py-2 border rounded"
    id="signup-localisation"
    name="localisation"
    type="text"
    onChange={formik.handleChange}
    onBlur={formik.handleBlur}
    value={formik.values.localisation}
  />
  {formik.touched.localisation &&
  formik.errors.localisation ? (
    <div className="text-red-500 text-sm">
      {formik.errors.localisation}
    </div>
  ) : null}
</div>
```

Le bouton “Inscription” soumet le formulaire, et fait apparaître une popup de confirmation.



La partie Frontend - “La Vue” étant expliquée, passons maintenant à la partie Backend, “Le Contrôleur” et “Le Modèle”

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Comme évoqué précédemment le back-end a été développé avec le SGBDR **MySQL**, administré avec l'outil **phpMyAmdin** via des requêtes SQL.

Quant à **Express**, il fonctionne avec des middleware c'est à dire des fonctions qui ont accès à l'objet request (req), l'objet response (res), et à la fonction next dans le cycle de requête-réponse de l'application. Ils peuvent exécuter du code, modifier les objets de requête et de réponse, mettre fin au cycle de requête-réponse, et appeler la fonction next pour passer le contrôle au middleware (à la fonction) suivant dans la chaîne.

Donc une fois notre base de donnée Mingle crée, il a fallu communiquer avec elle via ces requêtes SQL. Nous avons développé les 4 opérations de base de toute application web : LE CRUD.

Cet acronyme représente 4 opérations : créer, lire, mettre à jour, supprimer (Create, Read, Update, Delete).

Je vais vous illustrer mon CRUD en me basant sur l'entité “**User**”, en lien avec tout ce dont j'ai eu la charge depuis le début du projet.

Le “Model” - “Les Data”

Nous avons utilisé une approche orientée objets, la POO (Programmation Orientée Objet). Cette approche utilise des objets pour représenter nos concepts ou nos entités.

La POO nous a fourni des principes et des structures ont facilité l'implémentation de l'architecture MVC (encapsulation, héritage, polymorphisme et abstraction). Les modèles, vues et contrôleurs en MVC bénéficient directement de ces concepts POO.

Commençons par la classe “**AbstractManager**”, qui sert de modèle pour gérer certaines opérations de base de données pour toutes nos tables. La classe fournit une abstraction pour les opérations “**Read**” et “**Delete**”, ce qui simplifie l'interaction avec la base de données.

Les détails de l'implémentation des méthodes de requête SQL sont encapsulés dans la classe “**AbstractManager**”, ce qui signifie que le reste de l'application n'a pas besoin de connaître les détails de l'exécution des requêtes SQL.

Cette classe va être étendue pour ajouter des méthodes spécifiques à une table particulière (on la retrouvera par exemple dans la classe “**UserManager**”)

```
class AbstractManager {
  constructor({ table }) {
    this.table = table;
  }
  find(id) {
    return this.database.query(select * from ${this.table}
    where id = ?, [
      id,
    ]);
  }
  findAll() {
    return this.database.query(select * from ${this.table});
  }
  delete(id) {
    return this.database.query(delete from ${this.table} where
    id = ?, [id]);
  }
  setDatabase(database) {
    this.database = database;
  }
}
module.exports = AbstractManager;
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Le “Model” - “Les Data”

Passons maintenant à une classe plus spécifique, le “**UserManager**”. Elle hérite des fonctionnalités de la classe “**AbstractManager**” grâce à “**require**”, et appelle le constructeur de “**AbstractManager**” avec la table “users” avec le mot clé “**super**”

```
const AbstractManager = require("../AbstractManager");
class UserManager extends AbstractManager {
  constructor() {
    super({ table: "users" });
  }
}
```

J’ai ensuite créé une méthode “**insert**” (Create) qui prend pour objet “users” en paramètre, et qui exécute une requête SQL pour insérer les données de cet objet dans la table “users”.

```
insert(users) {
  return this.database.query(`insert into ${this.table}
    (username, mail, localisation, user_pass, avatar, biographie,
    service_type, service_count) values (?, ?, ?, ?, ?, ?, ?, ?)`, [
    users.username,
    users.mail,
    users.localisation,
    users.user_pass,
    users.avatar,
    users.biographie,
    users.service_type,
    users.service_count
  ]);
}
```

Passons à la méthode “**update**”, qui met à jour les champs de la table “users” en fonction des propriétés présentes dans l’objet users passé en paramètre. Elle construit dynamiquement avec du conditionnel la requête SQL en ajoutant uniquement les champs qui existent dans l’objet users.

```
update(users) {
  const fields = [];
  const values = [];

  if (users.username) {
    fields.push("username = ?");
    values.push(users.username);
  }
  if (users.mail) {
    fields.push("mail = ?");
    values.push(users.mail);
  }
  ....

  values.push(users.id);
  const sql = `UPDATE ${this.table} SET ${fields.join(", ")}
    WHERE id = ?`;
  return this.database.query(sql, values);
}
```

Il a enfin fallu exporter la classe “**UserManager**” pour qu’elle puisse être utilisée dans d’autres fichiers.

```
module.exports = UserManager;
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Le “Controller” - “Le Cerveau”

Le contrôleur “UserController” gère les opérations CRUD et l'authentification des utilisateurs dans notre application Node.js, en lien avec la base de données Mingle. Il utilise les modèles définis dans “UserManager” et “AbstractManager”, ainsi que les bibliothèques `argon2` pour le hachage des mots de passe et `jsonwebtoken` pour la gestion des tokens JWT.

Le “Controller” - “Le Cerveau” - Méthodes CRUD

Les méthodes “**browse**” et “**read**” récupèrent tous les utilisateurs de la base de données ou un utilisateur spécifique par son ID. Ils utilisent les méthodes “**findAll**” ou “**find**” du `AbstractManager`. Ils renvoient des messages d'erreur si le ou les utilisateurs n'ont pas été trouvés.

Exemple avec “browse”

```
const browse = (req, res) => {
  models.user
    .findAll()
    .then([rows]) => {
      res.send(rows);
    })
    .catch((err) => {
      console.error(err);
      res.sendStatus(500);
    });
};
```

La méthode “**add**” ajoute un nouvel utilisateur (et donc lui crée un nouvel id). C'est aussi dans cette méthode que je me suis servie de la méthode `.hash` d'`argon2` pour hacher le mot de passe (`user_pass`) avant de l'enregistrer.

Elle utilise la méthode aussi “**insert**” du `UserManager` et renvoie un statut 201 ainsi que la localisation de l'utilisateur nouvellement créé, ou un statut 500 en cas d'erreur.

```
const add = async (req, res) => {
  const user = req.body;
  try {
    user.user_pass = await argon2.hash(user.user_pass);
    models.user
      .insert(user)
      .then([result]) => {
        res.location(`/users/${result.insertId}`)
        .sendStatus(201);
      })
      .catch((err) => {
        console.error(err);
        res.sendStatus(500);
      });
  } catch (err) {
    console.error(err);
    res.sendStatus(500);
  }
};
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Le “Controller” - “Le Cerveau” - Méthodes CRUD

La méthode “**edit**” met à jour un utilisateur spécifique. Elle utilise aussi la méthode .hash d’argon2 pour hacher le mot de passe (user_pass) s’il est fourni.

Elle utilise la méthode “**update**” du “**UserManager**” et renvoie un statut 204 si la mise à jour est réussie, un statut 404 si l’utilisateur n’est pas trouvé, ou un statut 500 en cas d’erreur.

La méthode “**destroy**” fonctionne de la même façon, utilisant la méthode “**delete**” du “**AbstractManager**”

Exemple avec “edit”

```
const edit = async (req, res) => {
  const user = req.body;
  user.id = parseInt(req.params.id, 10);
  if (user.user_pass) {
    try {
      user.user_pass = await argon2.hash(user.user_pass);
    } catch (err) {
      console.error(err);
      return res.sendStatus(500);
    }
  } else {
    delete user.user_pass;
  }
  models.user
    .update(user)
    .then((result) => {
      if (result.affectedRows === 0) {
        res.sendStatus(404);
      } else {
        res.sendStatus(204);
      }
    })
    .catch((err) => {
      console.error(err);
      res.sendStatus(500);
    });
};
```

Le “Controller” - “Le Cerveau” - Méthodes d'Authentification

Le “**UserController**” ne m’a pas servi juste pour implémenter les méthode du CRUD. Je l’ai aussi utilisé pour les méthodes d’authentification.

La méthode “**verifyPassword**” vérifie le mot de passe de l’utilisateur en utilisant argon2. Si le mot de passe est correct, elle crée un token JWT et le renvoie avec l’ID de l’utilisateur. enfin, elle renvoie un statut 401 si le mot de passe est incorrect, ou un statut 500 en cas d’erreur.

```
const verifyPassword = (req, res) => {
  argon2
    .verify(req.user.user_pass, req.body.user_pass)
    .then((isVerified) => {
      if (isVerified) {
        const payload = { sub: req.user.id };
        const token = jwt.sign(payload, process.env.JWT_SECRET);
        res.status(201).send({ token, userId: req.user.id,
toggle: process.env.APP_DECO });
      } else {
        return res.sendStatus(401);
      }
    })
    .catch((err) => {
      console.error(err);
      return res.sendStatus(500);
    });
};
```


04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Le “Controller” - “Le Cerveau” - Méthodes d'Authentification

La méthode “verifyToken” joue un rôle crucial dans la sécurisation des routes de en vérifiant les tokens JWT inclus dans les en-têtes d'autorisation des requêtes, et en utilisant le secret défini dans les variables d'environnement (fichier “.env”) . Elle vérifie la présence et le format de l'en-tête d'autorisation.

Si le token est valide, elle passe au middleware suivant , sinon renvoie un statut 401.

```
const verifyToken = (req, res, next) => {
  try {
    const authorizationHeader = req.get("Authorization");
    if (authorizationHeader == null) {
      throw new Error("Authorization header is missing");
    }
    const [type, token] = authorizationHeader.split(" ");

    if (type !== "Bearer") {
      throw new Error("Authorization header has not the 'Bearer' type");
    }
    req.payload = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    console.error(err);
    res.sendStatus(401);
  }
};
```

Il a fallu enfin exporter les méthodes (ou middleware) du “UserController” pour qu'elles puissent être utilisées dans d'autres parties de l'application, notamment dans les routes.

```
module.exports = {
  browse,
  read,
  edit,
  add,
  destroy,
  getUserByEmail,
  verifyPassword,
  verifyToken,
};
```

Ces méthodes ont été utilisé dans le frontend via les fonctions “axios”. Voici un rappel sur la suppression de profil utilisateur.

```
const handleDeleteProfile = async () => {
  try {
    const token = localStorage.getItem("token");
    const response = await axios.delete(
      `${import.meta.env.VITE_BACKEND_URL}/users/${userId}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      },
    );
  }
};
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Les Routes et la Sécurité

Voyons maintenant le système qui définit les routes pour une application Express.js en utilisant un router “**router.js**”. Encore une fois, nous allons nous concentrer sur la partie “**users**”.

Chaque ligne va importer un module de contrôleur qui va gérer les opérations CRUD pour mon entité “**users**”.

C’est aussi ici que j’ai implémenté “**validators**”, qui importe des fonctions de validation pour vérifier les données des utilisateurs avant de les insérer ou de les mettre à jour (fonctions déterminées dans un fichier à part). Cela m’a permis d’ajouter une sécurité sur mes routes côté backend, en plus de la sécurité frontend.

```
router.get("/users", userControllers.browse);
router.get("/users/:id", userControllers.read);
router.put("/users/:id", validateUser, userControllers.edit);
router.post("/users", validateUser, userControllers.add);
router.delete("/users/:id", userControllers.destroy);
```

Les fonctions de validation sont stockées dans le fichier “**validator.js**”. Je me suis servi de la librairie “**express-validator**”. J’ai utilisé “**validationResult**” pour extraire et vérifier les résultats des validations. Chaque validation est exécutée en utilisant **validation.run(req)**. Une fois terminé, **validationResult(req)** est utilisé pour récupérer les erreurs potentielles. Elles sont retournées sous forme de réponse JSON avec un statut HTTP 400. Sinon, **next()** est appelé pour passer au middleware suivant dans la chaîne.

```
const validateUser = (req, res, next) => {
  let validations = [
    body("username")
      .isLength({ min: 3 })
      .withMessage("Le nom d'utilisateur doit comporter au moins 3 caractères"),
    body("mail").isEmail().withMessage("Adresse mail invalide"),
    body("localisation")
      .optional()
      .isLength({ min: 2 })
      .withMessage("La ville doit comporter au moins 2 caractères"),
    ...
  ];
  if (req.method === "POST") {
    validations.push(
      body("user_pass")
        .isLength({ min: 6 })
        .withMessage("Le mot de passe doit comporter au moins 6 caractères")
        .matches(/!@#%&*(),.?":{}|<>\/)
        .withMessage(
          "Le mot de passe doit comporter au moins 1 caractère spécial"
        )
    );
  }
  // Appliquer les validations
  Promise.all(validations.map((validation) => validation.run(req))).then(() => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      console.log(errors.array());
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  });
};
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Les Routes et la Sécurité

Le token de sécurité et le hachage du mot de passe mis en place permettent eux aussi de protéger projet.

En effet, chaque requête concernant les utilisateur doit contenir dans son en-tête le token unique, si non un message d'erreur sera renvoyé et l'accès sera refusé.

L'utilisateur obtient le jeton en s'authentifiant lors du login. Le jeton est ensuite envoyé avec chaque requête à l'API.

```
'Authorization': `Bearer ${localStorage.getItem('token')}`
```

Amélioration de la sécurité sur une version 2

La prochaine étape (à voir dans une V.2) serait de rajouter des rôles (par exemple “admin” et “users”), afin de sécuriser les routes admin contenant des données sensibles ou des fonctionnalités particulières du site.

Exemple d'une route sécurisée

```
router.get("/user/protected", userControllers.browse)
```

Un grande menace que nous n'avons pas eu le temps de traiter serait l'attaque **CSRF** (Cross-Site Request Forgery).

C'est un type d'attaque qui force un utilisateur authentifié à exécuter des actions non souhaitées sur l'application web où il est actuellement authentifié (étant donné que nous stockons en plus le token actuel dans le local storage). Cela peut entraîner des actions malveillantes telles que la modification de paramètres de compte, l'exécution de transactions non autorisées ou la divulgation de données sensibles.

Nous pourrions implémenter une protection **CSRF** côté serveur avec **Express** intégrer ce mécanisme côté client avec **React**.

```
npm install csurf cookie-parser
// Côté server
const express = require('express');
const cookieParser = require('cookie-parser');
const csrf = require('csurf');
// Côté client
useEffect(() => {
  // Obtenir le token CSRF depuis le serveur
  axios.get('/csrf-token')
    .then(response => {
      setCsrfToken(response.data.csrfToken);
    })
    .catch(error => {
      console.error('Error fetching CSRF token:', error);
    });
}, []);
.....
```

04 – DÉVELOPPEMENT

Backend – “Le Modèle” et “Le Contrôleur”

Amélioration de la sécurité sur une version 2

Une autre menace pas totalement couverte est la faille XSS (Cross-Site Scripting).

C'est est une vulnérabilité de sécurité qui permet à un attaquant d'injecter des scripts malveillants dans des pages web consultées par d'autres utilisateurs. Ces scripts peuvent voler des informations sensibles, manipuler l'affichage du site, ou exécuter des actions au nom de l'utilisateur.

Nous avons déjà mis en place un premier niveau de sécurité avec express-validator et YUP, malheureusement insuffisant.

Voici comment nous pourrions procéder :



```
npm install helmet xss-clean
// Côté server
const express = require('express');
const helmet = require('helmet');
const xss = require('xss-clean');
const app = express();
app.use(helmet());
app.use(xss());
// Côté client
npm install dompurify
import DOMPurify from 'dompurify';
const CommentForm = () => {
  const [username, setUsername] = useState('');
  const [comment, setComment] = useState('');
  const handleSubmit = (e) => {
    e.preventDefault();
    const sanitizedUsername = DOMPurify.sanitize(username);
    const sanitizedComment = DOMPurify.sanitize(comment);
    .....
```

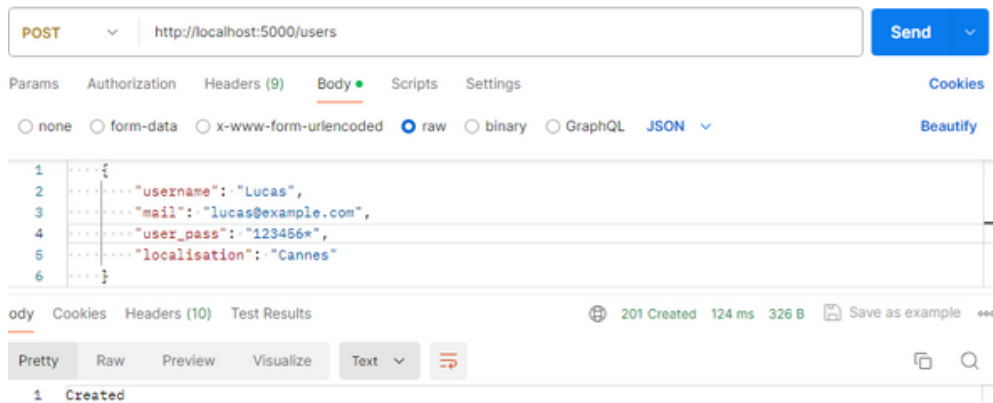
05 – FINALISATION DU PROJET

Test

Tout au long de la phase de développement, nous avons effectué différents types de tests pour s'assurer que l'application fonctionnait comme prévu.

Tout d'abord, nous avons testé notre **CRUD** avec **Postman**, afin de voir si toutes les routes étaient correctes.

Voici un exemple pour la requête POST, dans l'entité **"users"** :



Ensuite, nous avons mis en place des **tests unitaires**. Ils ont été réalisés de manière isolée sur un composant spécifique avant la mise en commun. Nous avons pour cela créé plusieurs branches de développement afin d'isoler le développement et le test. J'ai mis en place un test qui vérifie que les services notés dans l'édition de profil correspondent aux catégories intégrées dans l'application :



05 – FINALISATION DU PROJET

Test

J'ai rédigé mon test avec la méthode AAA (Arrange, Act, Assert) :

- Organiser les données nécessaires au test (Arrange)
- Exécuter le code que l'on souhaite tester (Act)
- Vérifier les résultats (Assert)

```
describe('validerServices', () => {
  it('should return true for valid services', () => {
    // Arrange
    const validServices = 'Aide au devoir, Aide au bricolage,
Aide au covoiturage';

    // Act
    const result = validerServices(validServices);

    // Assert
    expect(result).toBe(true);
  });

  it('should return false for invalid services', () => {
    // Arrange
    const invalidServices = 'Aide au devoir, Aide au jardinage';

    // Act
    const result = validerServices(invalidServices);

    ....
  });
});
```

Je me suis servie du framework Jest pour exécuter mon test :

```
PASS  __tests__/ServicesAutorises.test.js
validerServices
  ✓ should return true for valid services (6 ms)
  ✓ should return false for invalid services
  ✓ should return false when at least one service is invalid
  ✓ should return true for a single valid service (1 ms)
  ✓ should return false for a single invalid service
  ✓ should handle empty string
  ✓ should ignore leading and trailing whitespace (1 ms)
```

Enfin, nous avons réalisé des tests se rapprochant des test **end-to-end**, en simulant manuellement le comportement d'un utilisateur sur le site afin de vérifier le bon fonctionnement de l'ensemble des fonctionnalités du site.

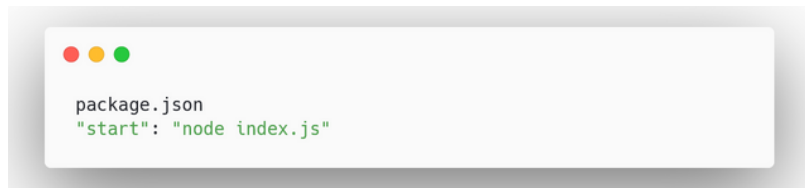
05 – FINALISATION DU PROJET

Déploiement

Nous avons choisi de déployer et d'héberger notre application (tant le frontend que le backend) sur [Render](#). La base de données reste quant à elle hébergée sur [AlwaysData](#).

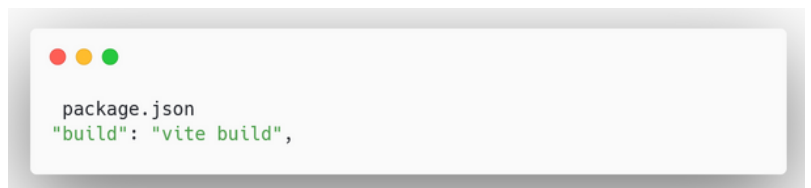
Avant de déployer l'application, nous avons vérifié toutes nos dépendances dans les fichiers "[package.json](#)", et nous avons du tester la version production grâce à [React preview](#) :

Côté backend nous avons lancé dans le terminal la commande "**npm run start**" pour lancer le serveur. Elle exécute le script défini sous la clé "start" dans le fichier "[package.json](#)", qui lance le serveur de production en utilisant Node.js avec le fichier `index.js`.

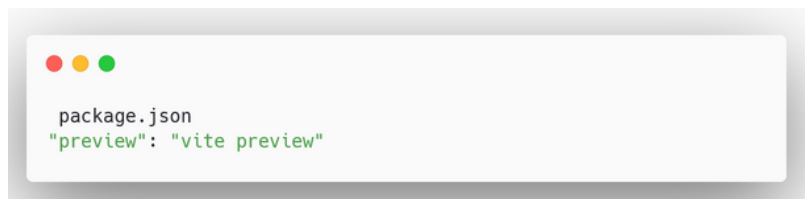


```
package.json
"start": "node index.js"
```

Côté frontend nous avons lancé la commande vite "**npm run build**" (qui exécute le script défini sous la clé "build" dans le fichier "[package.json](#)", et qui utilise Vite pour compiler et minifier le code source en une version prête pour la production), suivi de la commande "**npm run preview**" (qui exécute le script défini sous la clé "preview" dans le fichier "[package.json](#)", et qui lance un serveur pour servir les fichiers générés par la commande build).



```
package.json
"build": "vite build",
```



```
package.json
"preview": "vite preview"
```

Une fois fait, nous avons mergé notre branche "**dev**" sur la branche "**main**", et nous sommes passé au déploiement sur [Render](#). Le backend et le frontend ont été déployés séparément.

05 – FINALISATION DU PROJET

Déploiement

Pour le **backend** nous avons :

- Cliqué sur "+ New", et choisi "Web Service"
- Choisi notre répertoire GitHub <https://github.com/Emmanuelle-S/Mingle>
- Nommé le service "mingle-api"
- Pour "Root Directory" : tapé **"./backend"**
- Pour "Build Command" : tapé **"npm install"**
- Pour "Start Command" : tapé **"npm run start"**
- Dans "Environment Variables" : entré les variables d'environnement de `.backend/.env`

Suite à ces réglages nous avons pu déployer notre backend. Il a juste fallu revenir plus tard renseigner l'URL du frontend une fois déployé.

Une fois l'opération réussite, Nous avons vérifié que nous avons bien accès au données json de notre API.

Pour le **frontend** nous avons :

- Cliqué sur "+ New", et choisi "Static Site"
- Choisi notre répertoire GitHub <https://github.com/Emmanuelle-S/Mingle>
- Nommé le service "mingle-api"
- Pour "Root Directory" : tapé **"./frontend"**
- Pour "Build Command" : tapé **"npm install && npm run build"**
- Pour "Publish Directory" : tapé **"dist"**
- Dans "Environment Variables" : entré les variables d'environnement de `.frontend/.env`

Suite à ces réglages nous avons pu déployer notre frontend. Il a juste fallu revenir plus tard renseigner l'URL du backend une fois déployé.

Enfin, il a fallu effectué un dernier réglage dans les paramètres de redirection pour gérer le routage de React Router Dom. Nous avons dû renseigner dans la section "Redirection" :

- Source: /*
- Destination: /index.html
- Type: Rewrite

Une fois l'opération réussite, Nous avons vérifié que toutes les fonctionnalités étaient opérationnelles.

☐	Service name	Status	Type	Runtime	Region
☐	 Mingle	✓ Deployed	Static Site	Static	Global
☐	 mingle-api	✓ Deployed	Web Service	Node	Frankfurt

06 – DÉVELOPPEMENT DES COMPÉTENCES

Tout au long de la construction de ce projet j'ai effectuée diverses veilles afin d'approfondir mes compétences, et d'en apprendre de nouvelles. Je me suis notamment beaucoup servi des documentations des librairies que j'ai utilisé.

Exemple avec la doc de Tailwind



J'ai aussi sollicité nos formateurs, qui ont continué de nous faire avancer sur de nouvelles notions (ou d'en approfondir d'anciennes) afin de mener ce projet à bien.

07 – CONCLUSION ET ANNEXES

En conclusion, le projet MINGLE m'a permis de mettre en application toutes les compétences et notions apprises depuis décembre 2023. J'ai pu apprendre à travailler de façon fullstack, et cette mise en pratique m'a conforté dans l'idée de devenir développeuse web/Conceptrice d'application.

Le délais très serré de la conception de ce projet m'a obligé à être concentrée, efficace, autonome et autodidacte, et à énormément communiquer sur ce projet en groupe pour qu'on avance tous ensemble

Ce projet a été incroyablement stimulant car il était très ambitieux au vue du temps imparti, et il m'a donné envie de le continuer en approfondissant son développement sur les features et options que nous n'avons pas eu le temps de mettre en place.

07 – CONCLUSION ET ANNEXES

Remerciements

Je tiens tout d'abord à remercier Simplon, qui m'a offert l'opportunité d'intégrer cette formation, et de leur adaptabilité à ma situation particulière et imprévue.

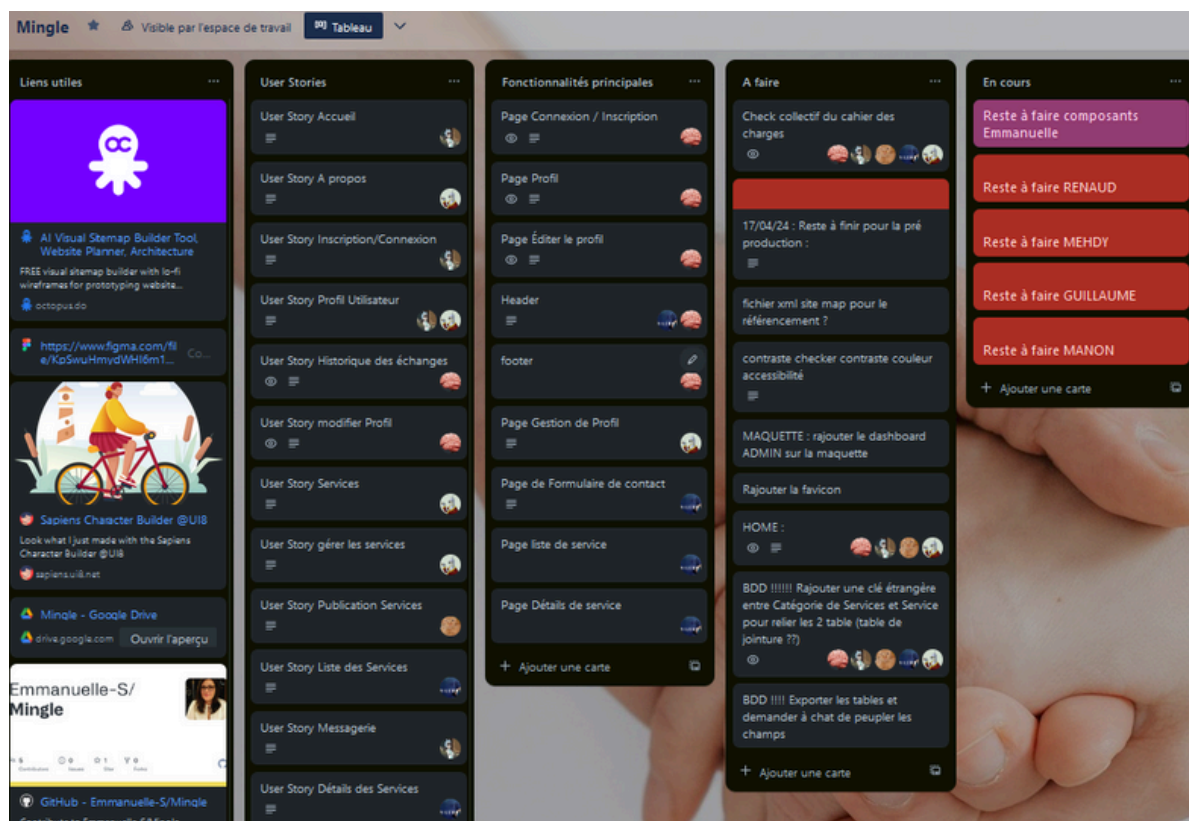
Je suis très reconnaissante envers nos formateurs, Salah, Jay, Nicolas et Adrien, ainsi que tous les intervenants, pour le temps qu'ils nous ont consacré, et leur implications dans la transmissions de leur savoir. Ils ont tous été très pédagogues, patients, humains, et ont été très impliqué dans notre réussite.

Merci à toute cette promotion 2024. Notre cohésion, notre entraide et nos moments de pause nécessaires ont grandement participé à la réussite de cette formation.

Enfin, merci à vous d'avoir pris le temps de lire ce dossier projet, et de m'accorder de votre temps pour le passage de cette certification.

Annexes

Capture d'une partie du Trello



07 – CONCLUSION ET ANNEXES

Annexes

Extrait de l'organisation d'un daily

DAILY

Add the Date 10/04/24	Mehdy	Manon	Renaud	Guillaume	Emmanuelle
Accomplissement d'hier	• Définition du projet • Mise en place des User Stories • Attribution des pages : Formulaire de contact + Liste de services + Détails des services	• Définition du projet • Création du Site Map • Mise en place des User Stories • Attribution des pages Publications des services + Politiques de confidentialités + Conditions d'utilisations	• Définition du projet • Mise en place des User Stories • Architecture du site • Organisation du Trello • Attribution des pages A propos + Gérer service + Services + 404	• Définition du projet • Mise en place de la majorité des User Stories • Réflexion sur l'architecture du site • Attribution des pages Accueil + Messagerie + Cookies + FAQ / Aide	• Définition du projet • Mise en place des User Stories • Création du Map Site • Attribution des pages Profil Utilisateur + Historique des échanges + Modifier profil + Inscription / Connexion
Tâches d'aujourd'hui	• Choisir une colorimétrie • Faire les wireframes • Commencer la maquette	• Choisir une colorimétrie • Faire les wireframes • Commencer la maquette	• Choisir une colorimétrie • Création du logo • Faire les wireframes • Commencer la maquette	• Choisir une colorimétrie • Faire les wireframes • Commencer la maquette • Vérifier les User Stories	• Choisir une colorimétrie • Création du suivi SCRUM • Faire les wireframes • Commencer la maquette
Points bloquants	• Pas de points bloquants	• Pas de points bloquants	• Pas de points bloquants	• Pas de points bloquants	• Pas de points bloquants

07 – CONCLUSION ET ANNEXES

Annexes

Extrait du cahier des charges

Présentation générale :

Présentation Générale :

Notre projet consiste en la conception et le développement d'une plateforme innovante de mise en relation entre particuliers, orientée en priorité étudiants. Cette plateforme collaborative ambitionne de répondre aux besoins croissants des particuliers et surtout des étudiants en matière d'échange de services au sein de la communauté, dans un contexte économique de plus en plus compliqué.

Contexte et Justification du Projet :

Le contexte actuel met en lumière un besoin pour un accès facile à des services spécifiques. Bien que de nombreuses plateformes existent déjà, elles ne répondent pas pleinement aux attentes en termes de convivialité, de personnalisation et de fiabilité. Notre projet vise à combler cette lacune en proposant une solution adaptée et efficace.

Objectifs du Projet :

Nos objectifs sont multiples. Nous aspirons à créer un environnement virtuel propice à l'échange de services prioritairement entre étudiants, qu'il s'agisse de compétences académiques, professionnelles ou personnelles. Nous visons également à encourager la collaboration et l'entraide au sein de la communauté générale, tout en favorisant le développement de réseaux sociaux et la création d'une communauté dynamique pour les utilisateurs.

Description de l'Audience Cible :

Notre audience cible comprend principalement les étudiants universitaires de tous niveaux et de toutes disciplines. Ces étudiants recherchent des solutions pratiques et accessibles pour répondre à leurs besoins variés, qu'il s'agisse de soutien académique, de services professionnels ou de simples échanges entre pairs. Notre plateforme vise à répondre à ces besoins de manière efficace et personnalisée. Notre projet n'est évidemment pas fermé à toute personne se proposant ou voulant demander un service, qu'elle soit étudiante ou non.

Fonctionnalités Principales :

La plateforme offrira une gamme de fonctionnalités essentielles pour faciliter les échanges de services. Parmi celles-ci, on retrouve la création de profils utilisateur détaillés. Les utilisateurs pourront publier des demandes de services spécifiques, tout en filtrant les offres en fonction de critères pertinents tels que la localisation géographique, le domaine d'expertise... Un système de messagerie intégré permettra aux utilisateurs de communiquer facilement pour discuter des détails des services proposés ou demandés. De plus, un système de certification par badge garantira la qualité des échanges sur la plateforme, renforçant ainsi la confiance au sein de la communauté.

Contraintes et Limitations :

Nous sommes conscients des contraintes et des limitations auxquelles nous pourrions être confrontés lors du développement de la plateforme. Celles-ci pourraient inclure des défis techniques, des limitations liées aux ressources humaines disponibles. Nous nous engageons à relever ces défis de manière proactive afin de garantir le succès et la pérennité de notre projet.

En résumé, notre plateforme de mise en relation représente une solution innovante pour favoriser la collaboration et l'entraide au sein de la communauté. En offrant un espace dédié à l'échange de services, nous visons à créer un réseau dynamique où chacun pourra trouver l'aide dont il a besoin tout en contribuant au succès de ses pairs.

Temporelles :

Le projet de développement de la plateforme collaborative pour étudiants est soumis à une contrainte temporelle importante avec une date limite fixée au 29 juillet 2024. Pour respecter cette échéance, le projet sera organisé en plusieurs sprints, chacun se concentrant sur des étapes spécifiques du processus de développement. Voici la planification des sprints envisagée :

1. Sprint 1 (Semaine 1-2) - Préparation et Planification :

- Choix du projet et constitution de l'équipe.
- Mise en place d'un tableau Trello pour la gestion de projet.
- Réflexion sur les fonctionnalités et les technologies à utiliser.
- Élaboration d'un plan de projet initial.

2. Sprint 2 (Semaine 3-4) - Définition des Besoins :

- Élaboration du cahier des charges détaillé, en prenant en compte les retours de l'équipe et des formateurs.
- Définition de la charte graphique de la plateforme.

3. Sprint 3 (Semaine 5-6) - Conception initiale :

- Création des premiers wireframes et maquettes de l'interface utilisateur.
- Réflexion sur la structure de la base de données avec la méthode MERISE.

→ Ces 3 sprints seront probablement réalisés en 2 semaines, au vu des contraintes de temps et des disponibilités de chacun. En effet, une période de stage de 6 semaines est prévue de fin avril à début juin.

4. Sprint 4 (Semaine 7-8) - Développement Initial :

- Mise en marche du développement de la plateforme en se concentrant sur les fonctionnalités essentielles.
- Implémentation des fonctionnalités de base identifiées dans le cahier des charges.

5. Sprint 5 (Semaine 9-10) - Consolidation et itération :

- Révision et ajustement des fonctionnalités existantes en fonction des retours de l'équipe et des premiers tests utilisateurs.
- Correction des éventuels bogues ou problèmes rencontrés lors du développement initial.

6. Sprint 6 (Semaine 11-12) - Finalisation et Tests :

- Finalisation du développement de la plateforme en ajoutant les fonctionnalités restantes et en peaufinant l'interface utilisateur.
- Tests approfondis de l'ensemble de la plateforme pour garantir sa stabilité, sa performance et sa sécurité.

7. Sprint 7 (Semaine 13-14) - Déploiement et Livraison :

- Préparation du déploiement de la plateforme sur les serveurs de production.
- Configuration des environnements de production et des sauvegardes.
- Déploiement de la plateforme sur les serveurs de production et vérification de son bon fonctionnement.

Fonctionnalités attendues :

Notre plateforme de mise en relation d'étudiants propose un ensemble de fonctionnalités conçues pour faciliter les échanges entre les membres de la communauté. Voici une synthèse des principales fonctionnalités attendues :

1. Création de Profil Utilisateur : Les utilisateurs auront la possibilité de créer des profils détaillés mettant en avant leurs compétences, leurs intérêts et leurs disponibilités. Ces profils serviront de vitrine pour les utilisateurs, facilitant ainsi la recherche de partenaires pour des collaborations spécifiques. Il y aura des profils privés avec possibilité de l'éditer et de le compléter. Une partie de ces profils sera visible de façon publique.

2. Publication de Demandes de Services : Les utilisateurs pourront publier des demandes de services spécifiques, qu'il s'agisse de soutien académique, de services professionnels ou de simples échanges entre pairs. Les demandes pourront être formulées de manière claire et concise, permettant ainsi aux autres membres de la communauté de répondre rapidement et efficacement.

3. Filtrage des Offres : Une fonctionnalité de filtrage avancé permettra aux utilisateurs de rechercher et de trier les offres en fonction de critères pertinents tels que la localisation géographique, le domaine d'expertise... Cela garantira une correspondance précise entre l'offre et la demande, facilitant ainsi la recherche de partenaires pour des collaborations spécifiques.

4. Messagerie intégrée : Un système de messagerie intégré permettra aux utilisateurs de communiquer facilement pour discuter des détails des services proposés ou demandés. Cette fonctionnalité favorisera les échanges et la coordination entre les membres de la communauté, garantissant ainsi une expérience fluide et efficace sur la plateforme.

5. Système de récompense : Un système de badge sera mis en place pour garantir la qualité des échanges sur la plateforme. Les utilisateurs auront la possibilité d'attribuer un point pour le service rendu, renforçant ainsi la confiance au sein de la communauté et facilitant la prise de décision pour les futurs partenariats.

Liste des fonctionnalités détaillées :

1. Authentification et Gestion des Comptes Utilisateurs :

- Les utilisateurs doivent pouvoir s'inscrire et créer un compte sur la plateforme.
- Les utilisateurs doivent pouvoir se connecter à leur compte à l'aide d'identifiants uniques (nom d'utilisateur ou adresse e-mail et mot de passe).
- Les utilisateurs doivent pouvoir modifier les informations de leur profil, y compris leurs compétences, leurs intérêts et leurs disponibilités.
- Les utilisateurs doivent pouvoir supprimer leur compte si nécessaire.

2. Publication et Gestion des Demandes de Services ou des Offres :

- Les utilisateurs doivent pouvoir publier des demandes ou des offres de services spécifiques en précisant le type de service recherché, les compétences requises et d'autres détails pertinents.
- Les utilisateurs doivent pouvoir modifier ou supprimer leurs demandes de services après leur publication.
- Les utilisateurs doivent pouvoir visualiser les demandes de services publiées par les autres membres de la plateforme.

3. Filtrage et Recherche de Demandes/Offres de Services :

- Les utilisateurs doivent pouvoir filtrer les demandes de services en fonction de critères pertinents tels que la localisation géographique, le domaine d'expertise ou le niveau d'études.
- Les utilisateurs doivent pouvoir effectuer une recherche par mots-clés pour trouver des demandes/offres de services spécifiques.

4. Messagerie et Communication entre Utilisateurs :

- Les utilisateurs doivent pouvoir envoyer des messages aux autres membres de la plateforme pour discuter des détails des services proposés ou demandés.
- Les utilisateurs doivent pouvoir visualiser et gérer leur historique de messages.

5. Notification et Alerte :

- Les utilisateurs doivent pouvoir recevoir des notifications et des alertes concernant les nouvelles demandes de services, les messages entrants et d'autres activités pertinentes sur la plateforme.

6. Gestion des Annonces et des Actualités :

- Les administrateurs doivent pouvoir publier des annonces et des actualités importantes pour informer les utilisateurs de mises à jour ou d'événements sur la plateforme.

7. Sécurité et Confidentialité :

- Le système doit garantir la sécurité des données des utilisateurs, y compris leur identité et leurs informations personnelles.
- Le système doit respecter les normes de confidentialité et de protection des données.

07 – CONCLUSION ET ANNEXES

Annexes

Exemple du wireframe pour la page Connexion/Inscription version desktop

The wireframe shows a desktop layout for a 'Connexion/Inscription' page. At the top, there is a dark header bar with a logo on the left and navigation links (Home, Service Category, Messaging, Logout, Account) on the right. Below the header, the main content area is divided into two columns. The left column is for 'Connexion' (Login) and contains fields for 'Email', 'Mot de passe' (Password), and 'Mot de passe oublié' (Forgot password), followed by a 'Se connecter' button. The right column is for 'Inscription' (Registration) and contains fields for 'Nom d'utilisateur' (Username), 'Email', 'Mot de passe', 'Confirmation du mot de passe', and a checkbox for 'J'accepte les termes et conditions', followed by an 'Inscription' button. At the bottom, there is a dark footer bar with links for 'Terms of Service', 'Privacy Policy', and 'Contact Us'.

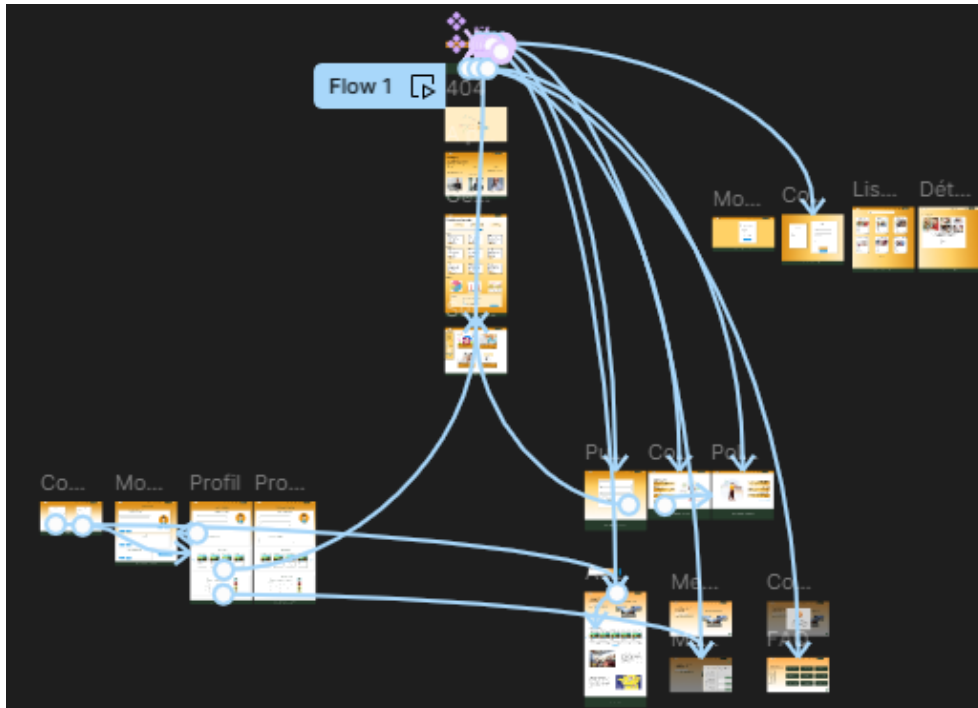
Exemple du wireframe pour la page Connexion/Inscription version mobile

The wireframe shows three mobile device screens for the 'Connexion/Inscription' page. Each screen has a dark header bar with a logo and a hamburger menu icon. The first screen, titled 'Android Large - Inscr...', shows the 'Inscription' form with fields for 'Nom d'utilisateur', 'Email', 'Mot de passe', 'Confirmation du mot de passe', and a checkbox for 'J'accepte les termes et conditions', followed by an 'Inscription' button. The second screen, titled 'Android large - Conn...', shows the 'Connexion' form with fields for 'Email', 'Mot de passe', and a 'Mot de passe oublié' link, followed by a 'Se connecter' button. The third screen, titled 'Android Large - Con...', shows a simplified view with 'Connexion' and 'Inscription' buttons. All three screens have a dark footer bar with links for 'Terms of Service', 'Privacy Policy', and 'Contact Us'.

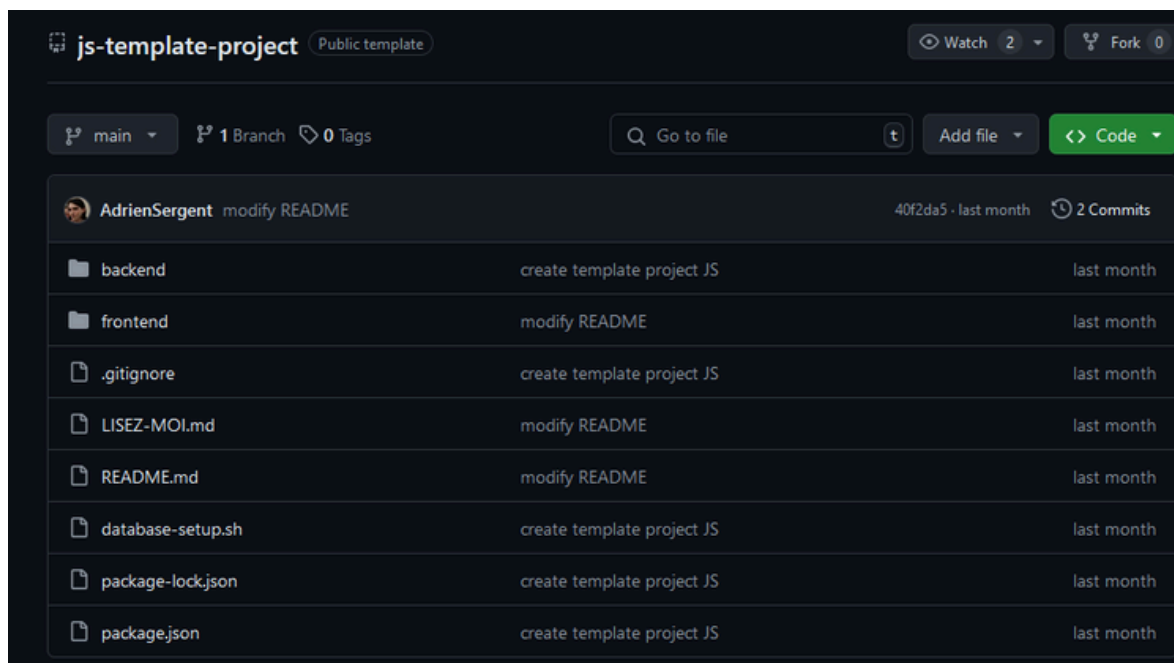
07 – CONCLUSION ET ANNEXES

Annexes

Prototypage de la maquette



Template de création du projet



07 – CONCLUSION ET ANNEXES

Annexes

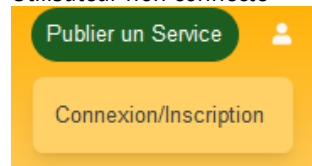
Configuration de Tailwind

```
const withMT = require("@material-tailwind/react/utils/withMT");

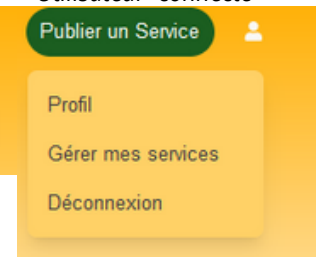
/** @type {import('tailwindcss').Config} */
module.exports = withMT({
  content: [
    "./index.html",
    "./src/**/*.vue,jsx,ts,tsx"
  ],
  theme: {
    extend: {
      colors: {
        black: '#0B1215',
        white: '#FAF9F6',
        primary: '#DD8F16',
        secondary: '#FFD166',
        light: '#FFEDC5',
        dark: '#263B2B',
        accent: '#0590E5',
        darkslategray: '#2F4F4F',
      },
    },
  },
  variants: {
    extend: {},
  },
  plugins: [],
});
```

*Affichage
conditionnel
suivant l'état que
connexion de
l'utilisateur*

Utilisateur non connecté



Utilisateur connecté



07 – CONCLUSION ET ANNEXES

Annexes

Voici le rendu de la partie Profil :

Rendu du Profil en version desktop

The desktop profile page for 'Mingle de Spidey' is divided into two main sections. The top section, 'Informations personnelles', contains input fields for 'Nom' (Spidey), 'Email' (spidey@spider.man), 'Ville' (Queens), and 'Type de Service Proposés' (Je suis un service). It also features a 'Photo de profil' placeholder and a 'Biographie' placeholder (Je suis une biographie). Below these are 'Éditer' and 'Supprimer' buttons. The bottom section, 'Services publiés', lists three services: 'Service 4' (Description du service 2, Status : activé), 'Service 6' (Description du service 6, Status : activé), and a placeholder 'dzdzdzcdcdcdcdcd' (dzdzdz, Status : activé). A 'Voir plus' button is at the bottom.

Rendu du Profil en version mobile

The mobile profile page for 'Mingle de Spidey' is a single-column layout. It starts with the title 'Profil Mingle de Spidey' and the section 'Informations personnelles'. It includes input fields for 'Nom' (Spidey), 'Email' (spidey@spider.man), 'Ville' (Queens), and 'Type de Service Proposés' (Je suis un service). Below these are a 'Photo de profil' placeholder and a 'Biographie' placeholder (Je suis une biographie). 'Éditer' and 'Supprimer' buttons are at the bottom of this section. The next section, 'Services publiés', lists 'Service 4' (Description du service 2, Status : activé) and 'Service 6' (Description du service 6, Status : activé). A blue circular button is visible at the bottom right.

Rendu de l'édition du Profil en version desktop

The desktop profile editing page is titled 'Editer le profil'. It has two columns. The left column, 'Informations personnelles', has input fields for 'Nom' (Spidey), 'Email' (spidey@spider.man), and 'Ville' (Queens). The right column, 'Photo de profil', has a 'Changer de photo de profil' label and a 'Parcourir...' button with the text 'Aucun fichier sélectionné.' Below these is a 'Biographie' section with a text area containing 'Je suis une biographie'. At the bottom is a 'Types de services proposés' section with a text area containing 'Je suis un service' and an 'Enregistrer' button.

07 – CONCLUSION ET ANNEXES

Annexes

Voici le rendu de la partie Connexion / Inscription :

Affichage version Desktop

The desktop version displays two side-by-side forms. The left form, titled 'Connexion', contains fields for 'Email' and 'Mot de passe', a link for 'Mot de passe oublié ?', and a blue 'Se connecter' button. The right form, titled 'Inscription', contains fields for 'Nom d'utilisateur', 'Email', 'Ville', 'Mot de passe', and 'Confirmation du mot de passe', a checkbox for 'J'accepte les termes et conditions', and a dark grey 'Inscription' button.

Affichage version Mobile avant de cliquer sur un des deux boutons

On a mobile screen, the two forms are stacked vertically. At the top, there are two large buttons: a blue 'Connexion' button and a dark grey 'Inscription' button.

Affichage version Mobile après avoir cliqué sur "Connexion"

After clicking the 'Connexion' button, the mobile screen displays the 'Connexion' form. It includes fields for 'Email' and 'Mot de passe', a link for 'Mot de passe oublié ?', and a blue 'Se connecter' button. The 'Inscription' button is no longer visible.